Data Mining: Frequent Pattern Mining

Fall 2026 (7761)

Dept. of Computer Science and Engineering
Instructor: Ting-Wei Lin

2026/10/16, Week 6

國立中興大學
NATIONAL CHUNG HSING UNIVERSITY

Definitions of association rules

- $I = \{i_1, i_2, \ldots, i_n\}$, the set of all items
- $T \subseteq I$, a transaction, represented as a nonempty itemset (each transaction is associated with an identifier, TID)
- $D = \{T_1, T_2, \ldots, T_m\}$, a transactional database
- k-itemset: an itemset containing k items
- Frequent k-itemset: support is at least the minimum support
- $X \rightarrow Y$, an association rule, where $X \subset I, Y \subset I, X \neq \emptyset, Y \neq \emptyset, X \bigcap Y = \emptyset$
  - Support: how common $(X \cup Y)$ is in the database
  - Confidence: how often Y occurs when X occurs

Apriori algorithm

- Level-wise approach (two-steps, join and prune)
  - Frequent k-itemsets are used to explore $(k + 1)$-itemsets
  - 1-itemsets → 2-itemtsets → 3-itemsets → …
- At each iteration k:
  ① Join step: Combine frequent k-itemsets to form candidate $(k + 1)$-itemsets:

$$C_{k+1} = L_k \bowtie L_k$$

  ② Prune step: Remove any candidate in C_{k+1} if one of its k-subsets is not in L_k, before counting its support.

min_sup count = 2

TID	ITEM
001	a, b, e
002	b, d
003	b, c
004	a, b, d
005	a, c
006	b, c
007	a, c, f
008	a, b, c, e
009	a, b, c

Scan database →

C_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2
$\{f\}$	1

count >= 2 →

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2
~~$\{f\}$~~	~~1~~

1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2

k=1 join

C_2

Itemset
$\{a, b\}$
$\{a, c\}$
$\{a, d\}$
$\{a, e\}$
$\{b, c\}$
$\{b, d\}$
$\{b, e\}$
$\{c, d\}$
$\{c, e\}$
$\{d, e\}$

k=1 prune

C_2

Itemset
$\{a, b\}$
$\{a, c\}$
$\{a, d\}$
$\{a, e\}$
$\{b, c\}$
$\{b, d\}$
$\{b, e\}$
$\{c, d\}$
$\{c, e\}$
$\{d, e\}$

count >= 2

L_2

Itemset	count
$\{a, b\}$	4
$\{a, c\}$	4
~~$\{a, d\}$~~	~~1~~
$\{a, e\}$	2
$\{b, c\}$	4
$\{b, d\}$	2
$\{b, e\}$	2
~~$\{c, d\}$~~	~~0~~
~~$\{c, e\}$~~	~~1~~
~~$\{d, e\}$~~	~~0~~

1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.

$\{c, e\}, \{c, d\}, \{d, e\}$ are not in L_2

L_2

Itemset	count
$\{a, b\}$	4
$\{a, c\}$	4
$\{a, e\}$	2
$\{b, c\}$	4
$\{b, d\}$	2
$\{b, e\}$	2

k=2 join

C_3

Itemset
$\{a, b, c\}$
$\{a, b, e\}$
$\{a, c, e\}$
$\{b, c, d\}$
$\{b, c, e\}$
$\{b, d, e\}$

k=2 prune

C_3

Itemset
$\{a, b, c\}$
$\{a, b, e\}$
~~$\{a, c, e\}$~~
~~$\{b, c, d\}$~~
~~$\{b, c, e\}$~~
~~$\{b, d, e\}$~~

count >= 2

L_3

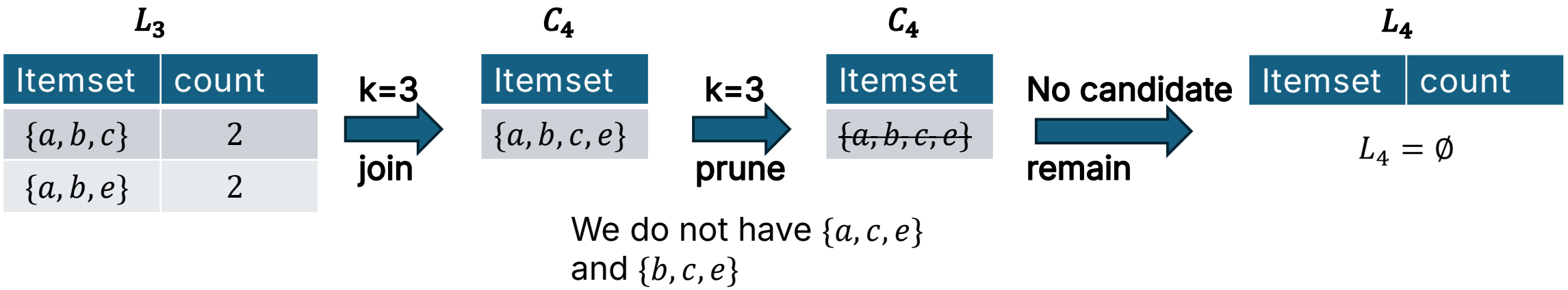
Itemset	count
$\{a, b, c\}$	2
$\{a, b, e\}$	2

- $\{a, b, c\} \rightarrow \{a, b\}, \{a, c\}, \{b, c\}$
- $\{a, b, e\} \rightarrow \{a, b\}, \{a, e\}, \{b, e\}$
- $\{a, c, e\} \rightarrow \{a, c\}, \{a, e\}, \{c, e\}$
- $\{b, c, d\} \rightarrow \{b, c\}, \{b, d\}, \{c, d\}$

  $\{b, c, e\} \rightarrow \{b, c\}, \{b, e\}, \{c, e\}$

  $\{b, d, e\} \rightarrow \{b, d\}, \{b, e\}, \{d, e\}$

1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.



1. Scan the database to find frequent 1-itemsets, L_1.
2. At iteration k:
   a. Generate candidate $(k+1)$-itemsets, C_{k+1}, from L_k.
   b. Prune candidates having an infrequent k-subset.
   c. Scan the database and retain candidates satisfying min_sup as L_{k+1}.
3. Stop when $L_{k+1} = \emptyset$.

L

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2

L_2

Itemset	count
$\{a,b\}$	4
$\{a,c\}$	4
$\{a,e\}$	2
$\{b,c\}$	4
$\{b,d\}$	2
$\{b,e\}$	2

L_3

Itemset	count
$\{a,b,c\}$	2
$\{a,b,e\}$	2

L_2 association rules:

$\{a,b\}$: subset $\{a\}$ and $\{b\}$ ➔ $\{a\} \rightarrow \{b\} \left(\frac{count(\{a,b\})}{count(\{a\})} = \frac{4}{6}\right), \{b\} \rightarrow \{a\} \left(\frac{4}{7}\right)$

$\{a,c\}$: subset $\{a\}$ and $\{c\}$ ➔ $\{a\} \rightarrow \{c\} \left(\frac{4}{6}\right), \{c\} \rightarrow \{a\} \left(\frac{4}{6}\right)$

$\{a,e\}$: subset $\{a\}$ and $\{e\}$ ➔ $\{a\} \rightarrow \{e\} \left(\frac{2}{6}\right), \{e\} \rightarrow \{a\} \left(\frac{2}{2}\right)$

$\{b,c\}$: subset $\{b\}$ and $\{c\}$ ➔ $\{b\} \rightarrow \{c\} \left(\frac{4}{7}\right), \{c\} \rightarrow \{b\} \left(\frac{4}{6}\right)$

$\{b,d\}$: subset $\{b\}$ and $\{d\}$ ➔ $\{b\} \rightarrow \{d\} \left(\frac{2}{7}\right), \{d\} \rightarrow \{b\} \left(\frac{2}{2}\right)$

$\{b,e\}$: subset $\{b\}$ and $\{e\}$ ➔ $\{b\} \rightarrow \{e\} \left(\frac{2}{7}\right), \{e\} \rightarrow \{b\} \left(\frac{2}{2}\right)$

L

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2

L_2

Itemset	count
$\{a, b\}$	4
$\{a, c\}$	4
$\{a, e\}$	2
$\{b, c\}$	4
$\{b, d\}$	2
$\{b, e\}$	2

L_3

Itemset	count
$\{a, b, c\}$	2
$\{a, b, e\}$	2

L_3 association rules:

$\{a, b, c\}$: subset $\{a\}, \{b\}, \{c\}, \{a, b\}, \{a, c\}, \{b, c\}$

$$\{a\} \rightarrow \{b, c\}\left(\frac{2}{6}\right), \{\mathrm{b}\} \rightarrow \{\mathrm{a}, \mathrm{c}\}\left(\frac{2}{7}\right), \{\mathrm{c}\} \rightarrow \{\mathrm{a}, \mathrm{b}\}\left(\frac{2}{6}\right), \{\mathrm{a}, \mathrm{b}\} \rightarrow \{\mathrm{c}\}\left(\frac{2}{4}\right), \{\mathrm{a}, \mathrm{c}\} \rightarrow \{b\}\left(\frac{2}{4}\right), \{b, c\} \rightarrow \{a\}\left(\frac{2}{4}\right)$$

$\{a, b, e\}$: subset $\{a\}, \{b\}, \{e\}, \{a, b\}, \{a, e\}, \{b, e\}$

$$\{a\} \rightarrow \{b, e\}\left(\frac{2}{6}\right), \{\mathrm{b}\} \rightarrow \{\mathrm{a}, \mathrm{e}\}\left(\frac{2}{7}\right), \{\mathrm{e}\} \rightarrow \{\mathrm{a}, \mathrm{b}\}\left(\frac{2}{2}\right), \{\mathrm{a}, \mathrm{b}\} \rightarrow \{\mathrm{e}\}\left(\frac{2}{4}\right), \{\mathrm{a}, \mathrm{e}\} \rightarrow \{b\}\left(\frac{2}{2}\right), \{b, e\} \rightarrow \{a\}\left(\frac{2}{2}\right)$$

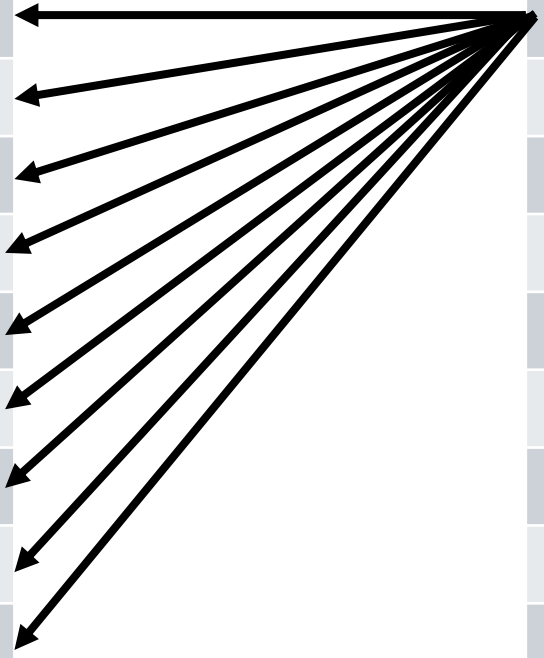
Improving the efficiency of Apriori

① Apriori optimizations
- Hash-based technique
- Transaction reduction
- Partitioning
- Sampling

② Beyond Apriori
- FP-Tree

Support Counting

TID	ITEM
001	a, b, e +1
002	b, d
003	b, c
004	a, b, d +1
005	a, c
006	b, c
007	a, c
008	a, b, c, e+1
009	a, b, c +1



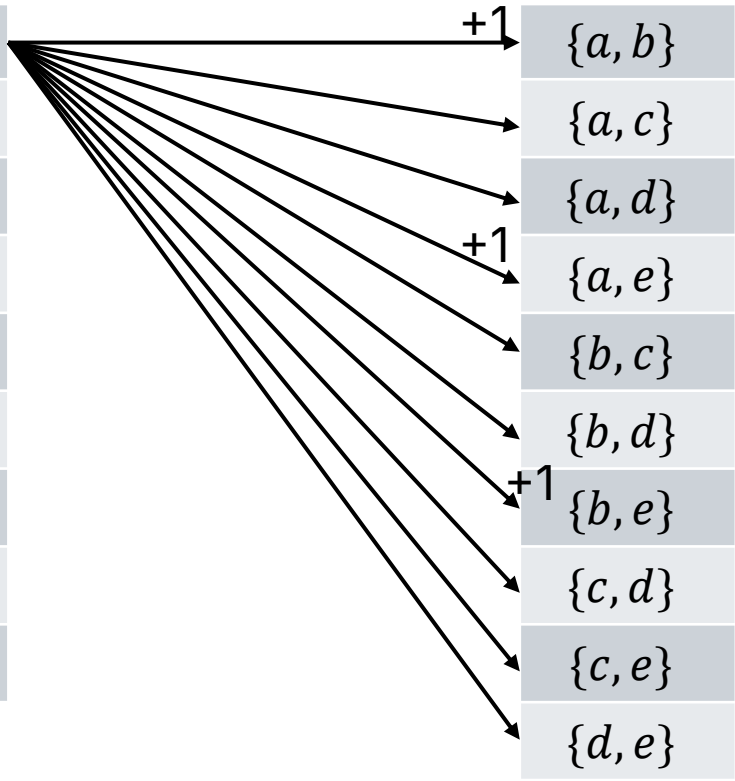
Itemset
$\{a, b\}$
$\{a, c\}$
$\{a, d\}$
$\{a, e\}$
$\{b, c\}$
$\{b, d\}$
$\{b, e\}$
$\{c, d\}$
$\{c, e\}$
$\{d, e\}$

Candidate-driven counting: For each candidate, scan all transactions.

Support Counting

$T01$: $\{a, b, e\} \Rightarrow \{a, b\}, \{a, e\}, \{b, e\}$

TID	ITEM
001	a, b, e
002	b, d
003	b, c
004	a, b, d
005	a, c
006	b, c
007	a, c
008	a, b, c, e
009	a, b, c



Transaction-driven counting: For each transaction, find its candidate subsets.

Hash-based technique

- Direct Hashing and Pruning (DHP), 1995 (PCY Algorithm)

An Effective Hash-Based Algorithm for Mining Association Rules

Jong Soo Park*, Ming-Syan Chen and Philip S. Yu

IBM Thomas J. Watson Research Center
Yorktown Heights, New York 10598
{c1park, mschen, psyu}@watson.ibm.com

Abstract

In this paper, we examine the issue of mining association rules among items in a large database of sales transactions. The mining of association rules can be mapped into the problem of discovering large itemsets where a large itemset is a group of items which appear in a sufficient number of transactions. The problem of discovering large itemsets

bar-code technology has made it possible for retail organizations to collect and store massive amounts of sales data. Catalog companies can also collect sales data from the orders they receive. A record in such data typically consists of the transaction date, the items bought in that transaction, and possibly also customer-id if such a transaction is made via the use of a credit card or

Problem in Candidate Generation

C_2

Itemset
$\{a, b\}$
$\{a, c\}$
$\{a, d\}$
$\{a, e\}$
$\{b, c\}$
$\{b, d\}$
$\{b, e\}$
$\{c, d\}$
$\{c, e\}$
$\{d, e\}$

C_3

Itemset
$\{a, b, c\}$
$\{a, b, e\}$
$\{a, c, e\}$
$\{b, c, d\}$
$\{b, c, e\}$
$\{b, d, e\}$

C_4

Itemset
$\{a, b, c, e\}$

Direct Hashing and Pruning (DHP)

- Why is Apriori expensive?
  - Candidate sets can be very large, especially C_2 .

$$|C_2| = \binom{m}{2}, m = |L_1|$$

  - Support counting requires scanning a large transaction database at every level.
- Initial candidate set generation is the key issue to improve
- Amount of transaction data must be scanned

Need to much time on C_2!!!

Direct Hashing and Pruning (DHP)

- Key ideas of DHP
  - Hash $k+1$-itemsets observed during the scan to prune unlikely candidates early.
  - Reduce the number of candidates whose support must be counted.
  - Reduce the transaction database for later passes.

Using hashing to filter out unqualified candidates

TID	Items
100	A, C, D
200	B, C, E
300	A, B, C, E
400	B, E

$\{A, C\}, \{A, D\}, \{C, D\}$
$\{B, C\}, \{B, E\}, \{C, E\}$
$\{A, B\}, \{A, C\}, \{A, E\}, \{B, C\}, \{B, E\}, \{C, E\}$
$\{B, E\}$

	$\{C, E\}$ $\{C, E\}$ $\{A, D\}$	$\{A, E\}$	$\{B, C\}$ $\{B, C\}$		$\{B, E\}$ $\{B, E\}$ $\{B, E\}$	$\{A, B\}$	$\{A, C\}$ $\{C, D\}$ $\{A, C\}$
Hash table	3	1	2	0	3	1	3
address	0	1	2	3	4	5	6

$$h(\{x, y\}) = ((ord(x) * 10 + ord(y)) \; mod \; 7$$

Using hashing to filter out unqualified candidates

	$\{C,E\}$ $\{C,E\}$ $\{A,D\}$	$\{A,E\}$	$\{B,C\}$ $\{B,C\}$		$\{B,E\}$ $\{B,E\}$ $\{B,E\}$	$\{A,B\}$	$\{A,C\}$ $\{C,D\}$ $\{A,C\}$
Hash table	3	1	2	0	3	1	3
address	0	1	2	3	4	5	6

TID	Items
100	A,C,D
200	B,C,E
300	A,B,C,E
400	B,E

$\min_sup = 2$

L_1

Itemset
$\{A\}$
$\{B\}$
$\{C\}$
$\{E\}$

Join

C_2

Itemset
$\{A,B\}$
$\{A,C\}$
$\{A,E\}$
$\{B,C\}$
$\{B,E\}$
$\{C,E\}$

hash

C_2

Itemset
$\{A,C\}$
$\{B,C\}$
$\{B,E\}$
$\{C,E\}$

Support

L_2

Itemset
$\{A,C\}$
$\{B,C\}$
$\{B,E\}$
$\{C,E\}$

$h(\{x,y\}) = ((ord(x) * 10 + ord(y)) \; mod \; 7$

Transaction Reduction for the Next Pass

$\min_sup = 2$

TID	Items
100	A, C, D
200	B, C, E
300	A, B, C, E
400	B, E

$\{A, C\}, \{A, D\}, \{C, D\}$

$\{B, C\}, \{B, E\}, \{C, E\}$

$\{A, B\}, \{A, C\}, \{A, E\}, \{B, C\}, \{B, E\}, \{C, E\}$

$\{B, E\}$

L_2

Itemset
$\{A, C\}$
$\{B, C\}$
$\{B, E\}$
$\{C, E\}$

Retain only items participating in L_2 . $(T_{ID} \cap L_2)$

Item A: 1 $a[0] = A = 1$ Item B: 2 $a[1] = \mathrm{B} = 2$ Item C: 3 $a[2] = C = 3$ Item E: 2 $a[3] = E = 2$

TID: 300, Keep: B,C,E and Discard: A

For the next pass $k = 2 \rightarrow 3$, discard an item if it appears in fewer than k frequent k-itemsets within the transaction.

Transaction Reduction for the Next Pass

$\min_sup = 2$

TID	Items
100	A, C, D
200	B, C, E
300	A, B, C, E
400	B, E

$\{A, C\}, \{A, D\}, \{C, D\}$

$\{B, C\}, \{B, E\}, \{C, E\}$

$\{A, B\}, \{A, C\}, \{A, E\}, \{B, C\}, \{B, E\}, \{C, E\}$

$\{B, E\}$

L_2

Itemset
$\{A, C\}$
$\{B, C\}$
$\{B, E\}$
$\{C, E\}$

Retain only items participating in candidates in C_2 .

Item A: 1 $a[0] = 1$

Item C: 1 $a[1] = 1$

Item D: 0 $a[2] = 0$

→ Discard TID: 100

Table 3: Comparison of execution time (T15.I4.D100)

	Apriori	DHP	
	number	number	D_k, $\lvert D_k \rvert$
L_1	820	820	6,700KB, 100,000
C_2	335,790	338	6,700KB, 100,000
L_2	207	207	
C_3	618	618	659KB, 20,602
L_3	201	201	
C_4	184	184	546KB, 17,417
L_4	98	98	
C_5	30	30	332KB, 10,149
L_5	23	23	
C_6	1	1	24KB, 756
L_6	1	1	
total time	39.39	13.91	

Transaction reduction

- Prune items and transactions that cannot contribute to future frequent itemsets.
- Reducing the number of transactions scanned in future iterations
- A transaction that does not contain any candidate $(k+1)$-itemsets can be removed before the next pass
  - Such a transaction can be marked or removed from further consideration

Partitioning Approach

- Partitioning the data to find candidate itemsets
  - Any globally frequent itemset must be locally frequent in at least one partition.
- The transaction database is divided into partitions that fit in main memory.
  - Partition size is chosen to be resident in main memory
  - Local minimum support count

Partitioning Approach (cont.)

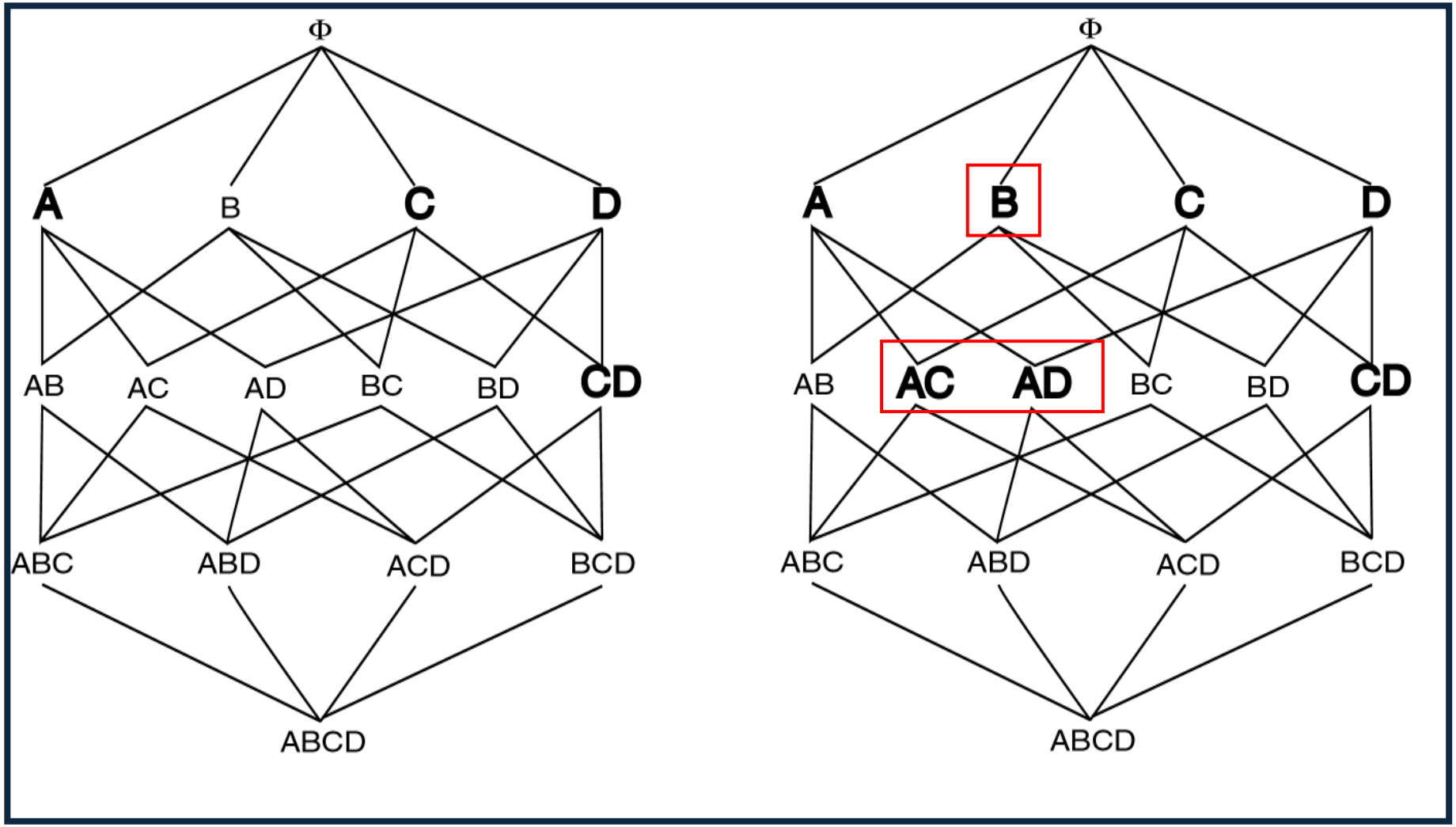
- An Efficient Algorithm for Mining Association Rules in Large Databases, 1995, VLDB (SON Algorithm)
- Two database scans
  1. First scan: generate a set of all potentially frequent itemsets
     - Each partition generates the local frequent itemsets
     - Local frequent itemsets are generated by itemset-record_ID data structure
  2. Second scan: actual support is measured
     - Collection of local frequent itemset = global candidate set
     - Global frequent itemsets are found by scan DB

Sampling Approach

- Mine frequent itemsets from a random sample of the database, then validate them against the full database
  1. Pick a random sample S of transaction database D (S is chosen to be resident in main memory)
  2. Search for frequent itemsets (L_S) in S instead of full database D
  3. Compute actual support by scanning the database to validate the sampled candidates and their negative border.
     - $NB = \{X \notin L_s | \forall Y \subset X, Y \in L_s\}$: All itemsets X such that X is not frequent, but every subset of X is frequent
  4. If not all frequent itemsets are found, second scan of D

Sampling Large Databases for Association Rules, 1996

Sampling Approach



Potentially Large Itemset (Ls)

$\{A\}, \{C\}, \{D\}, \{C, D\}$

Negative Border (NB):

$\{B\}\ \{A, C\}, \{A, D\}$

If $\{B\}, \{A, C\}, \{A, D\}$ turns to be frequent

- sampling failed

Frequent-Pattern (FP) Growth

Mine frequent itemsets without explicit candidate generation

Mining Frequent Patterns without Candidate Generation *

Jiawei Han, Jian Pei, and Yiwen Yin
School of Computing Science
Simon Fraser University
{han, peijian, yiweny}@cs.sfu.ca

ACM SIGMOD, 2000

Abstract

Mining frequent patterns in transaction databases, time-series databases, and many other kinds of databases has been studied popularly in data mining research. Most of the previous studies adopt an Apriori-like candidate set generation-and-test approach. However, candidate set generation is still costly, especially when there exist prolific patterns and/or long patterns.

In this study, we propose a novel frequent pattern tree (FP-tree) structure, which is an extended prefix-tree structure for storing compressed, crucial information about frequent patterns, and develop an efficient FP-tree-based mining method, FP-growth, for mining *the complete set of frequent patterns* by pattern fragment growth. Efficiency of mining is achieved with three techniques: (1) a large database is compressed into a highly condensed, much smaller data structure, which avoids costly, repeated database scans, (2) our FP-tree-based mining adopts a pattern fragment growth method to avoid the costly generation of a large number of candidate sets, and (3)

1 Introduction

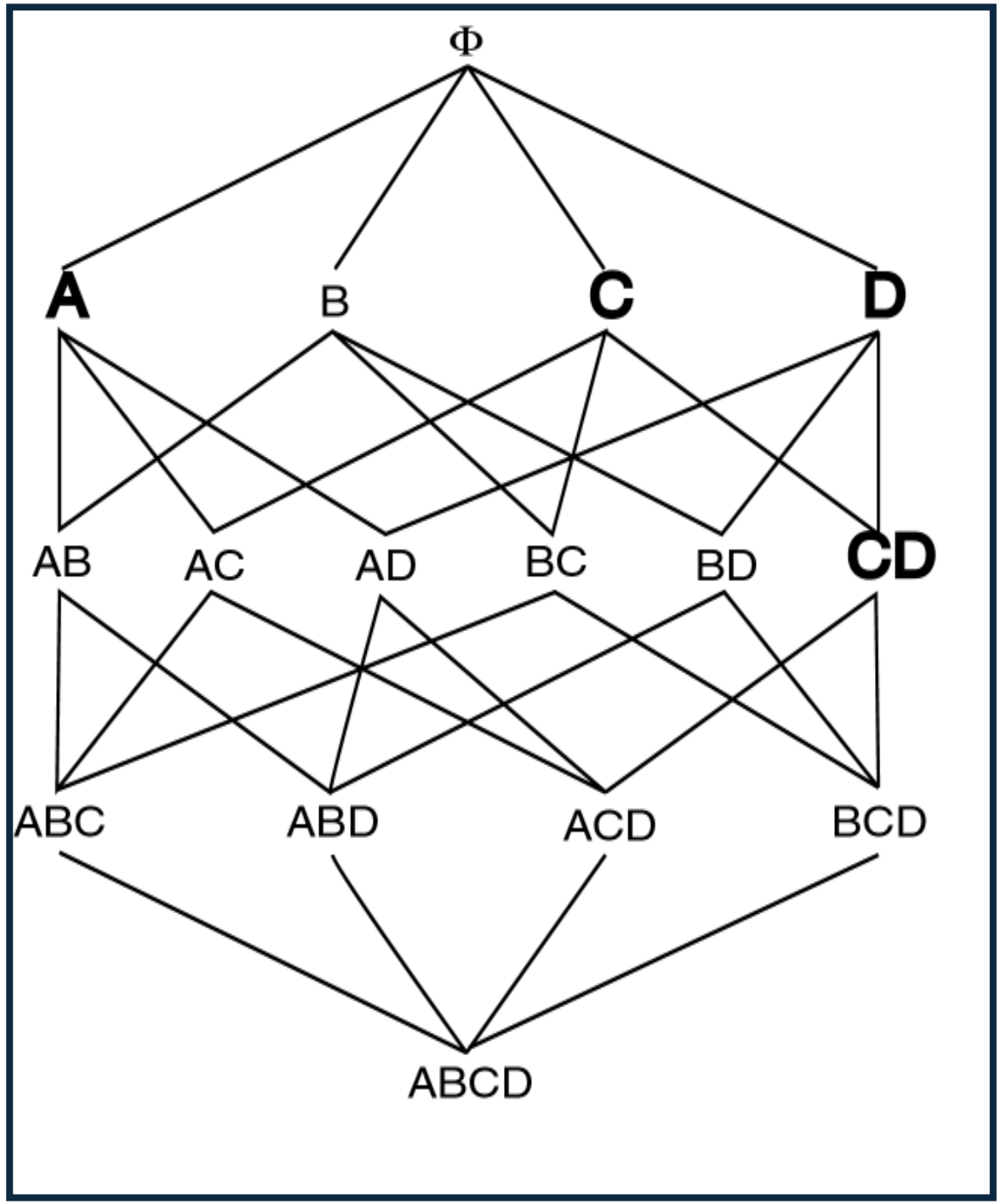
Frequent pattern mining plays an essential role in mining associations [3, 12], correlations [6], causality [19], sequential patterns [4], episodes [14], multi-dimensional patterns [13, 11], max-patterns [5], partial periodicity [9], emerging patterns [7], and many other important data mining tasks.

Most of the previous studies, such as [3, 12, 18, 16, 13, 17, 20, 15, 8], adopt an Apriori-like approach, which is based on an *anti-monotone* Apriori *heuristic* [3]: *if any length k pattern is not frequent in the database, its length $(k+1)$ super-pattern can never be frequent.* The essential idea is to iteratively generate the set of candidate patterns of length $(k+1)$ from the set of frequent patterns of length k (for $k \geq 1$), and check their corresponding occurrence frequencies in the database.

The Apriori heuristic achieves good performance gain by (possibly significantly) reducing the size

Mining Frequent Patterns by Pattern Growth

- Apriori: A breadth-first search mining algorithm
  - First find the complete set of frequent k-itemsets
  - Generate $(k+1)$-itemset candidates from frequent k-itemsets
  - Scan DB again to find true frequent $(k+1)$-itemsets
- Problem in Apriori
  - Need to generate a huge number of candidate sets (especially in C_2)
  - Need to repeatedly scan the database and count the support of the remaining candidates.
    - In each pass, prune candidates first, then scan the database to count the support of the remaining candidates.


Φ
A
B
C
D
AB
AC
AD
BC
BD
CD
ABC
ABD
ACD
BCD
ABCD

FP-Growth Approach

- Motivation for a different mining methodology
  - Can we develop a depth-first search mining algorithm that mines the complete set of frequent item sets without such a costly candidate generation process?
  - For a frequent pattern p, can we restrict the search for its extensions to transactions containing p?
- This idea leads to the frequent-pattern growth approach.

FP-Growth Approach: Example

$\min_sup = 2$

TID	Items
100	A, B, D
200	B, C
300	A, B, C
400	A, C

Frequent pattern $p = \{B\}$

Restrict the search for its extensions to transactions containing $\{B\}$

- Transactions: 100, 200, 300
  - Possible extensions of $\{B\}$:
    $\{A, B\}, \{B, D\}, \{B, C\}, \{A, B, D\}, \{A, B, C\}$
  - Frequent Extensions of $\{B\}$:
    $\{A, B\}, \{B, \mathrm{C}\}$

Once we focus on pattern $\{B\}$, transactions not containing B can never support an extension of $\{B\}$. FP-Growth exploits this locality recursively.

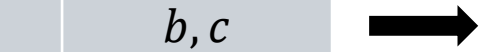
FP-Growth Overview

- Divide-and-Conquer strategy
  1. Construct FP-Tree
     - Compress the database representing frequent items to retain the itemset association information
     - After removing infrequent items and sorting the remaining items by global frequency
  2. FP-Growth
     - Transactions sharing a common prefix share the same path in the FP-tree.
     - For each frequent suffix item p:
       ① Collect its conditional pattern base.
       ② Construct its conditional FP-tree.
       ③ Recursively grow frequent patterns ending in p.

Construct FP-Tree

- Step 1: Find frequent 1-itemset (the same as Apriori), sorted items in frequency descending order by scanning DB

TID	ITEM
001	a, b, e
002	b, d
003	b, c
004	a, b, d
005	a, c
006	b, c
007	a, c, f
008	a, b, c, e
009	a, b, c



Itemset	count
$\{b\}$	7
$\{a\}$	6
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2
$\{f\}$	1

F-list: b-a-c-d-e

Find frequent 1-items, but sort by count

minimum support count = 2

Construct FP-Tree (cont.)

- Step 2: Scan DB and construct the FP-tree and let items in each transaction sort according to descending support count

Itemset	count
$\{b\}$	7
$\{a\}$	6
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2

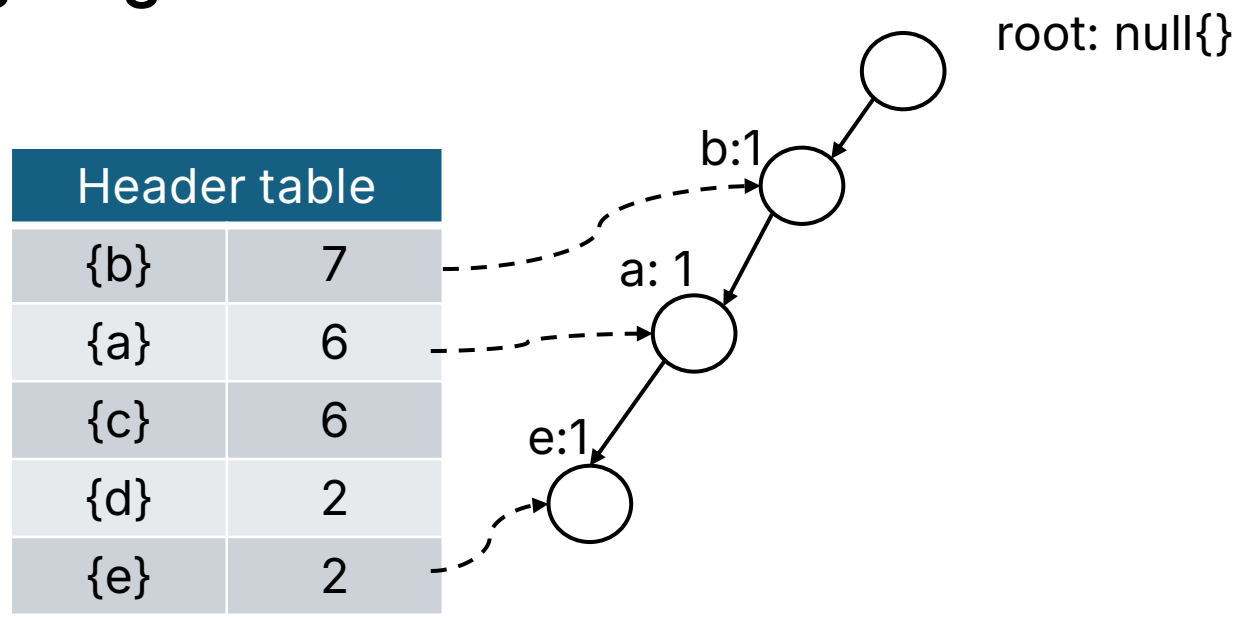
F-list: b-a-c-d-e

TID	ITEM	Ordered
001	a, b, e	b, a, e
002	b, d	b, d
003	b, c	b, c
004	a, b, d	b, a, d
005	a, c	a, c
006	b, c	b, c
007	a, c, f	a, c
008	a, b, c, e	b, a, c, e
009	a, b, c	b, a, c

Construct the FP-tree by aggregating the ordered set with common prefix

TID	ITEM	Ordered
001	a, b, e	b, a, e
002	b, d	b, d
003	b, c	b, c
004	a, b, d	b, a, d
005	a, c	a, c
006	b, c	b, c
007	a, c	a, c
008	a, b, c, e	b, a, c, e
009	a, b, c	b, a, c

Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2

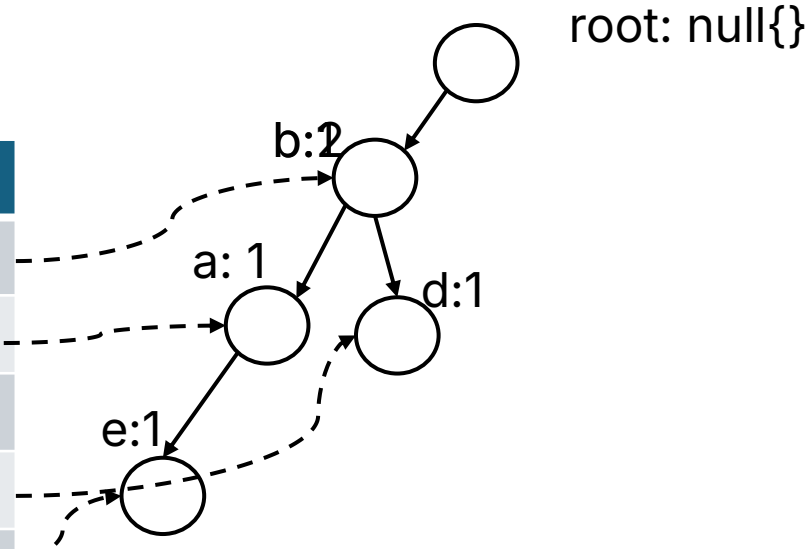


For each transaction, insert the ordered frequent itemlist into an FP-tree, with shared sub-branches merged, counts accumulated

Construct the FP-tree by aggregating the ordered set with common prefix

TID	ITEM	Ordered
001	a, b, e	b, a, e
002	b, d	b, d
003	b, c	b, c
004	a, b, d	b, a, d
005	a, c	a, c
006	b, c	b, c
007	a, c	a, c
008	a, b, c, e	b, a, c, e
009	a, b, c	b, a, c

Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2

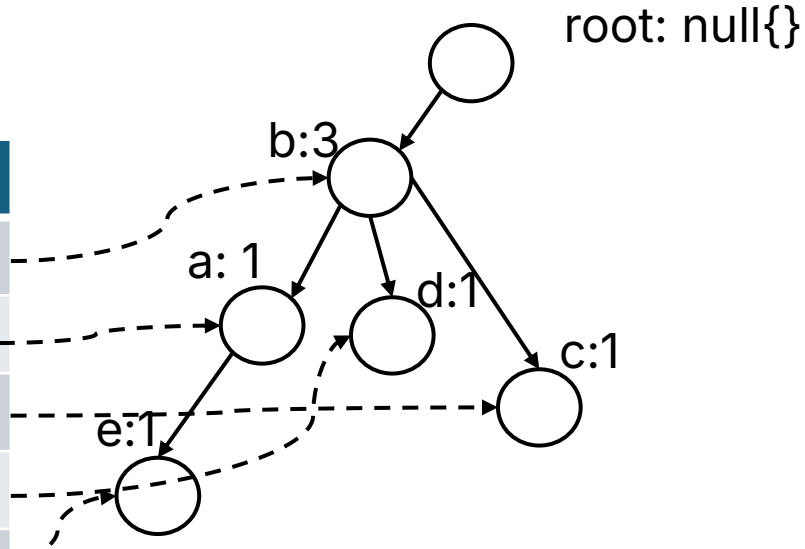


For each transaction, insert the ordered frequent itemlist into an FP-tree, with shared sub-branches merged, counts accumulated

Construct the FP-tree by aggregating the ordered set with common prefix

TID	ITEM	Ordered
001	a, b, e	b, a, e
002	b, d	b, d
003	b, c	b, c
004	a, b, d	b, a, d
005	a, c	a, c
006	b, c	b, c
007	a, c	a, c
008	a, b, c, e	b, a, c, e
009	a, b, c	b, a, c

Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2

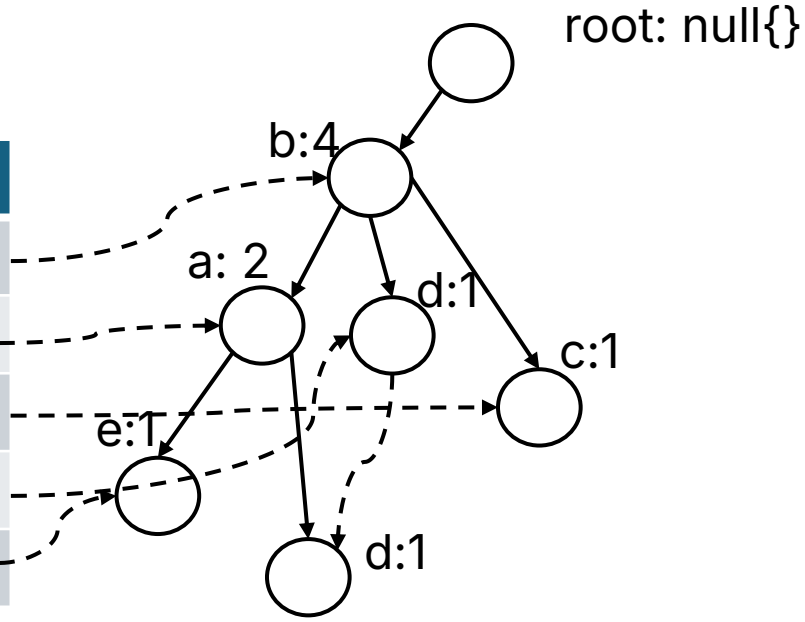


For each transaction, insert the ordered frequent itemlist into an FP-tree, with shared sub-branches merged, counts accumulated

Construct the FP-tree by aggregating the ordered set with common prefix

TID	ITEM	Ordered
001	a, b, e	b, a, e
002	b, d	b, d
003	b, c	b, c
004	a, b, d	b, a, d
005	a, c	a, c
006	b, c	b, c
007	a, c	a, c
008	a, b, c, e	b, a, c, e
009	a, b, c	b, a, c

Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2

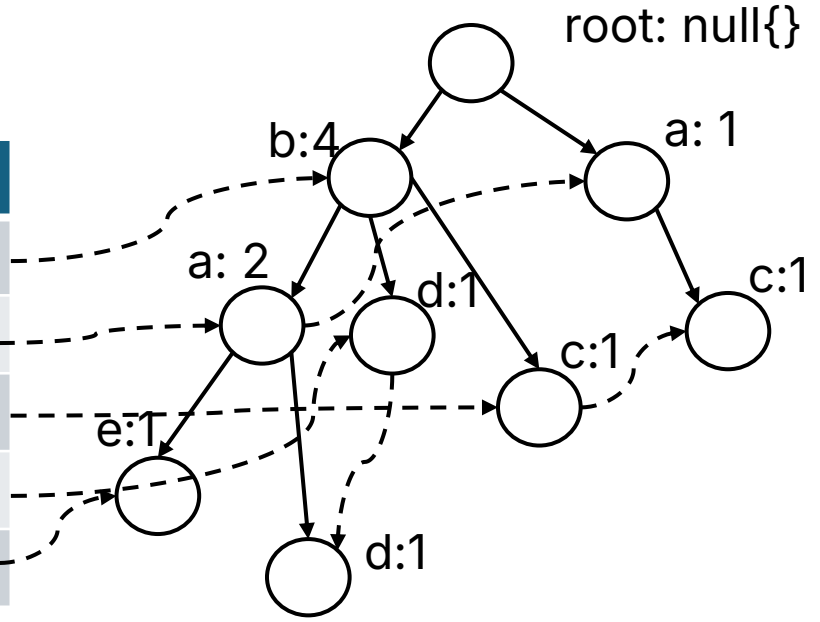


For each transaction, insert the ordered frequent itemlist into an FP-tree, with shared sub-branches merged, counts accumulated

Construct the FP-tree by aggregating the ordered set with common prefix

TID	ITEM	Ordered
001	a, b, e	b, a, e
002	b, d	b, d
003	b, c	b, c
004	a, b, d	b, a, d
005	a, c	a, c
006	b, c	b, c
007	a, c	a, c
008	a, b, c, e	b, a, c, e
009	a, b, c	b, a, c

Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2

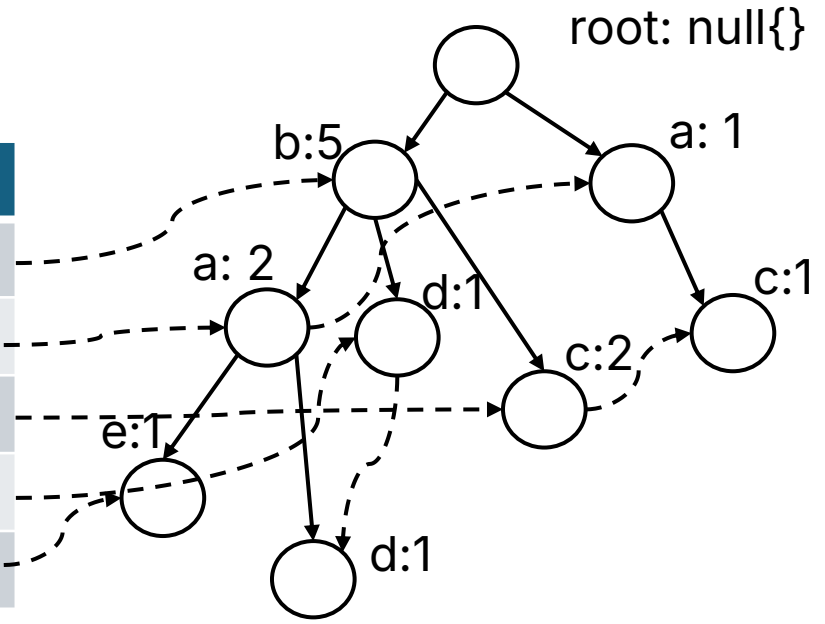


For each transaction, insert the ordered frequent itemlist into an FP-tree, with shared sub-branches merged, counts accumulated

Construct the FP-tree by aggregating the ordered set with common prefix

TID	ITEM	Ordered
001	a, b, e	b, a, e
002	b, d	b, d
003	b, c	b, c
004	a, b, d	b, a, d
005	a, c	a, c
006	b, c	b, c
007	a, c	a, c
008	a, b, c, e	b, a, c, e
009	a, b, c	b, a, c

Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2

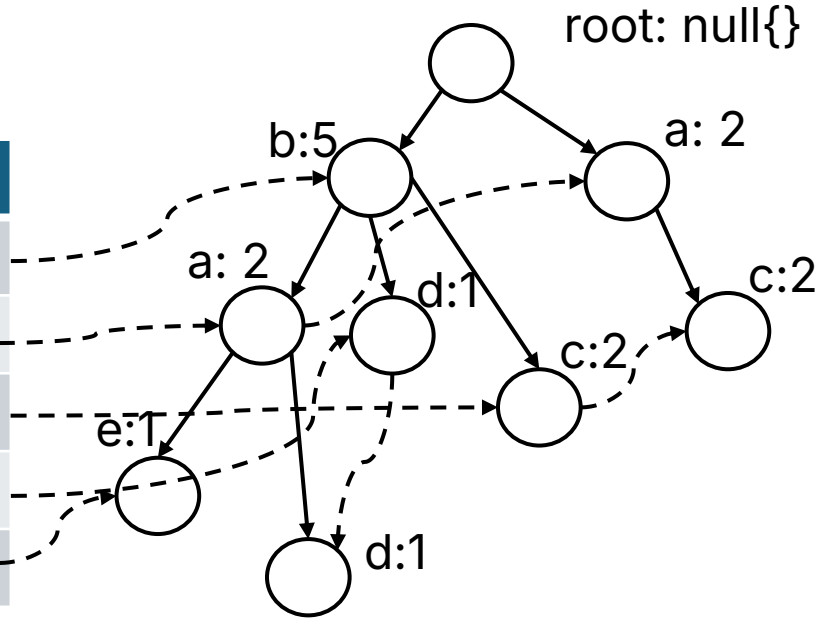


For each transaction, insert the ordered frequent itemlist into an FP-tree, with shared sub-branches merged, counts accumulated

Construct the FP-tree by aggregating the ordered set with common prefix

TID	ITEM	Ordered
001	a, b, e	b, a, e
002	b, d	b, d
003	b, c	b, c
004	a, b, d	b, a, d
005	a, c	a, c
006	b, c	b, c
007	a, c	a, c
008	a, b, c, e	b, a, c, e
009	a, b, c	b, a, c

Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2

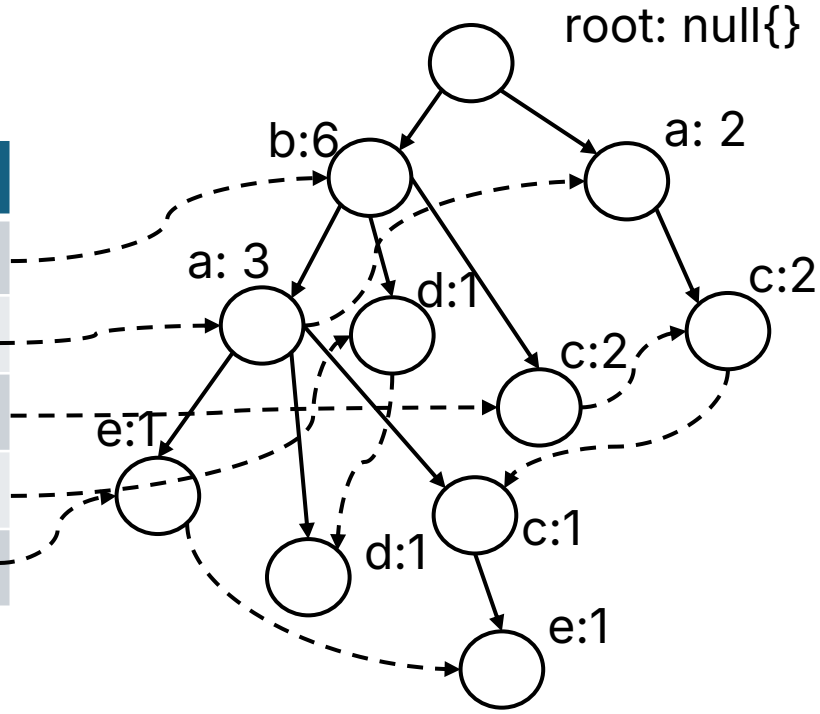


For each transaction, insert the ordered frequent itemlist into an FP-tree, with shared sub-branches merged, counts accumulated

Construct the FP-tree by aggregating the ordered set with common prefix

TID	ITEM	Ordered
001	a, b, e	b, a, e
002	b, d	b, d
003	b, c	b, c
004	a, b, d	b, a, d
005	a, c	a, c
006	b, c	b, c
007	a, c	a, c
008	a, b, c, e	b, a, c, e
009	a, b, c	b, a, c

Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2

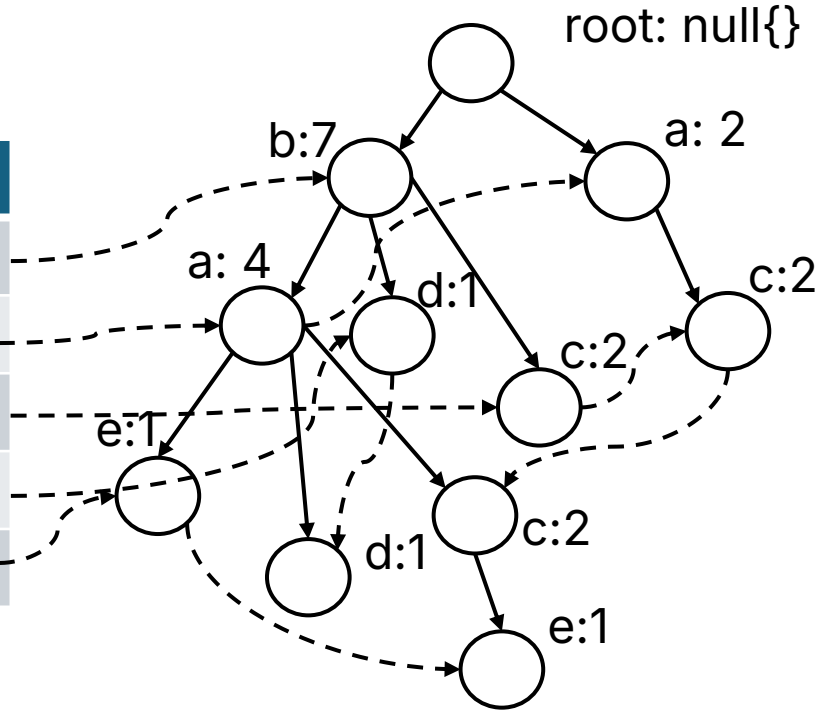


For each transaction, insert the ordered frequent itemlist into an FP-tree, with shared sub-branches merged, counts accumulated

Construct the FP-tree by aggregating the ordered set with common prefix

TID	ITEM	Ordered
001	a, b, e	b, a, e
002	b, d	b, d
003	b, c	b, c
004	a, b, d	b, a, d
005	a, c	a, c
006	b, c	b, c
007	a, c	a, c
008	a, b, c, e	b, a, c, e
009	a, b, c	b, a, c

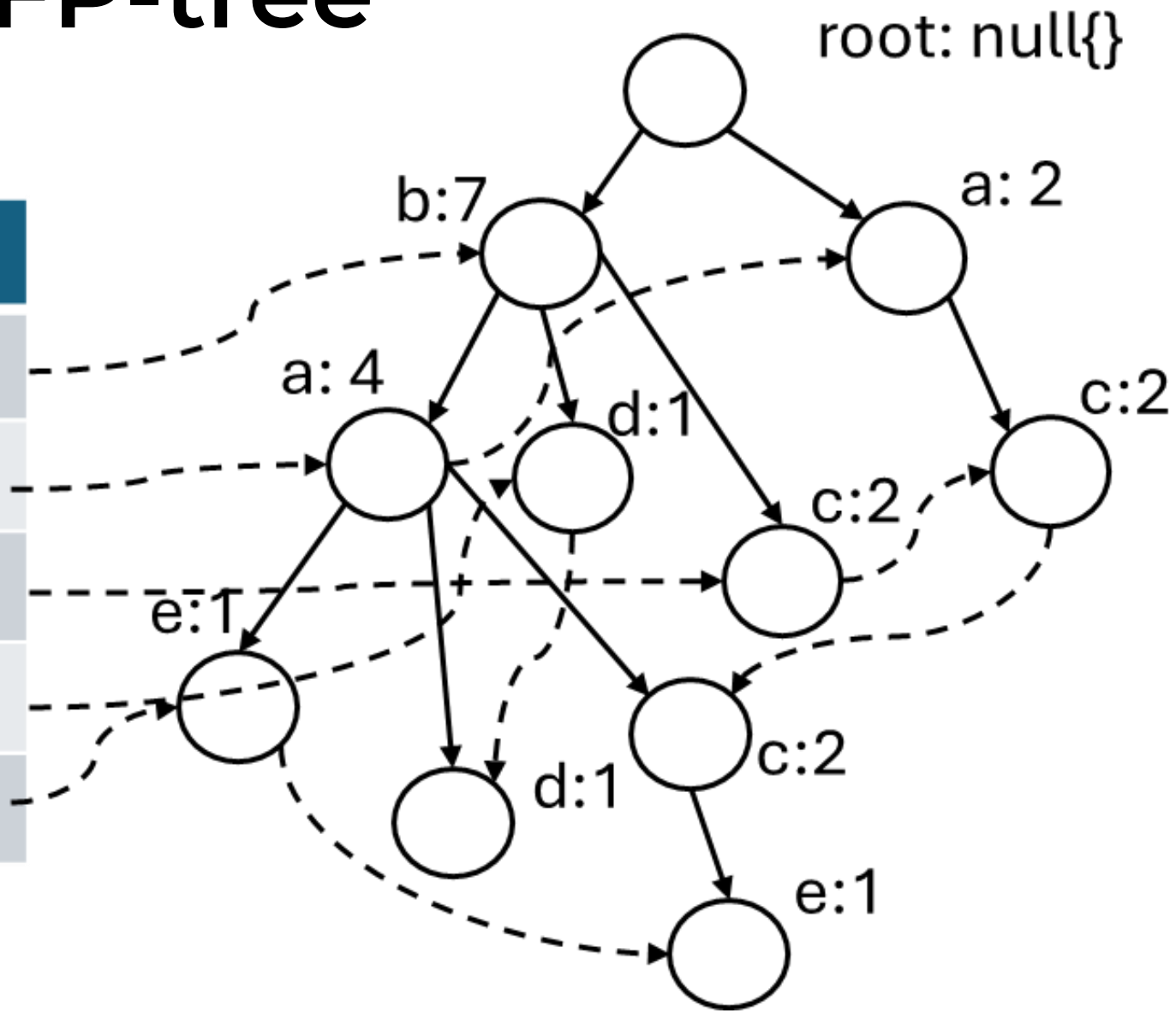
Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2



For each transaction, insert the ordered frequent itemlist into an FP-tree, with shared sub-branches merged, counts accumulated

Completed FP-tree

Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2



FP-Tree Another Example

TID	Items in the Transaction
100	$\{f, a, c, d, g, i, m, p\}$
200	$\{a, b, c, f, l, m, o\}$
300	$\{b, f, h, j, o, w\}$
400	$\{b, c, k, s, p\}$
500	$\{a, f, c, e, l, p, m, n\}$

f:4, a:3, c:4, b:3, m:3, p:3

F-list = f-c-a-b-m-p

TID	Items in the Transaction	Ordered, frequent itemlist
100	$\{f, a, c, d, g, i, m, p\}$	f, c, a, m, p
200	$\{a, b, c, f, l, m, o\}$	f, c, a, b, m
300	$\{b, f, h, j, o, w\}$	f, b
400	$\{b, c, k, s, p\}$	c, b, p
500	$\{a, f, c, e, l, p, m, n\}$	f, c, a, m, p

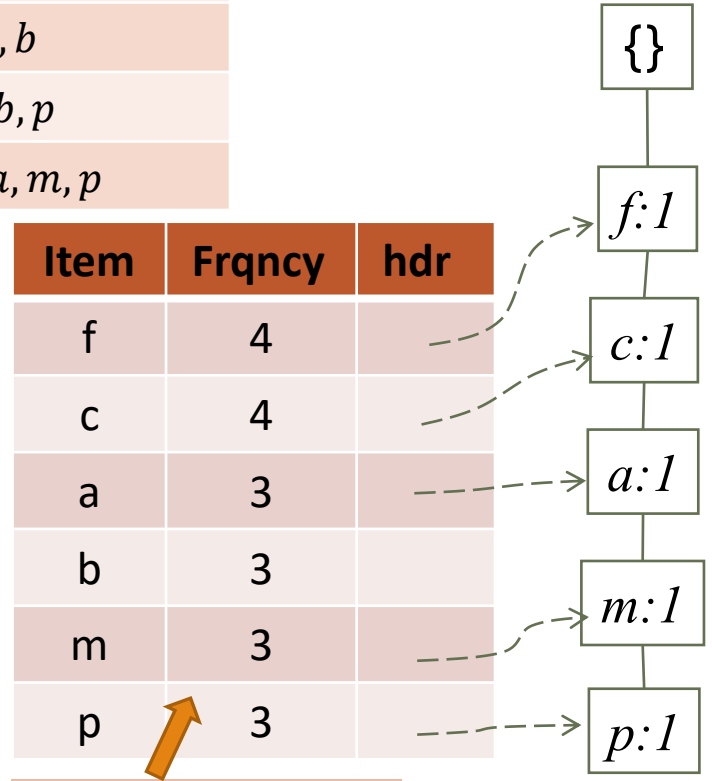
FP-Tree Another Example

TID	Ordered, frequent itemlist
100	f, c, a, m, p
200	f, c, a, b, m
300	f, b
400	c, b, p
500	f, c, a, m, p

After inserting the 1st frequent Itemlist: *"f, c, a, m, p"*

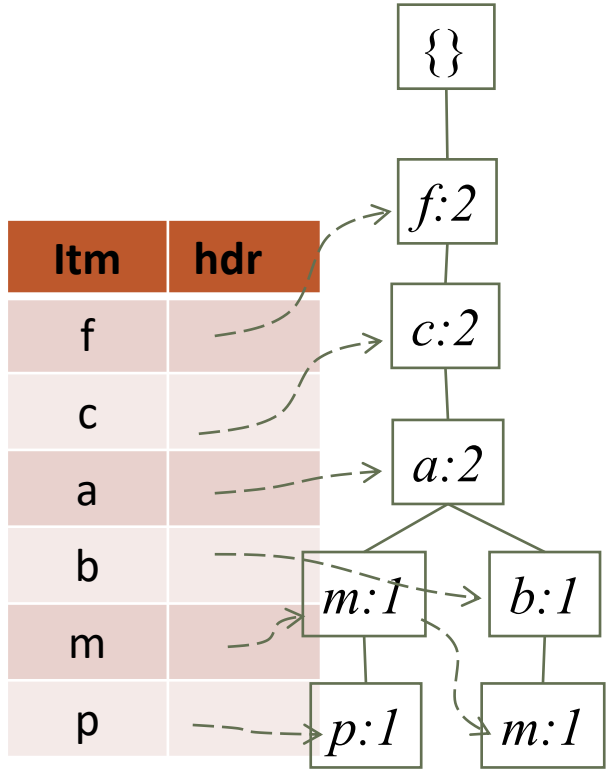
Item	Frqncy	hdr
f	4	
c	4	
a	3	
b	3	
m	3	
p	3	

Header Table



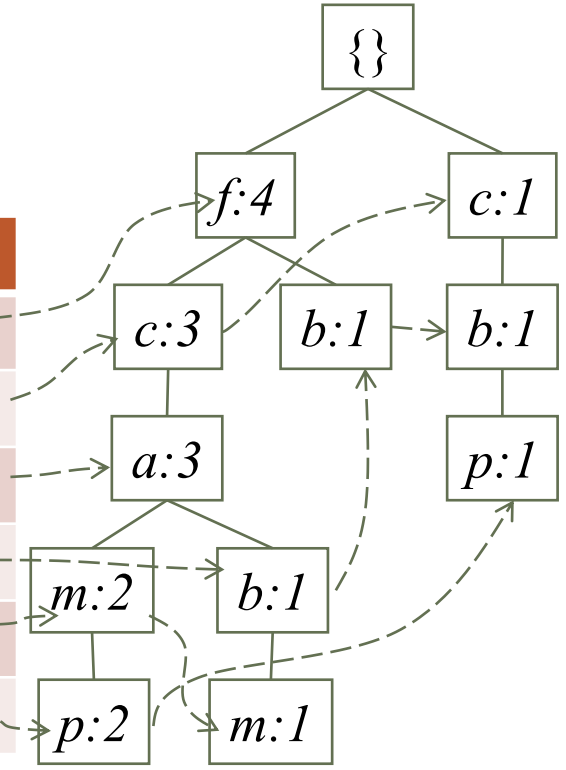
After inserting the 2nd frequent itemlist *"f, c, a, b, m"*

Itm	hdr
f	
c	
a	
b	
m	
p	



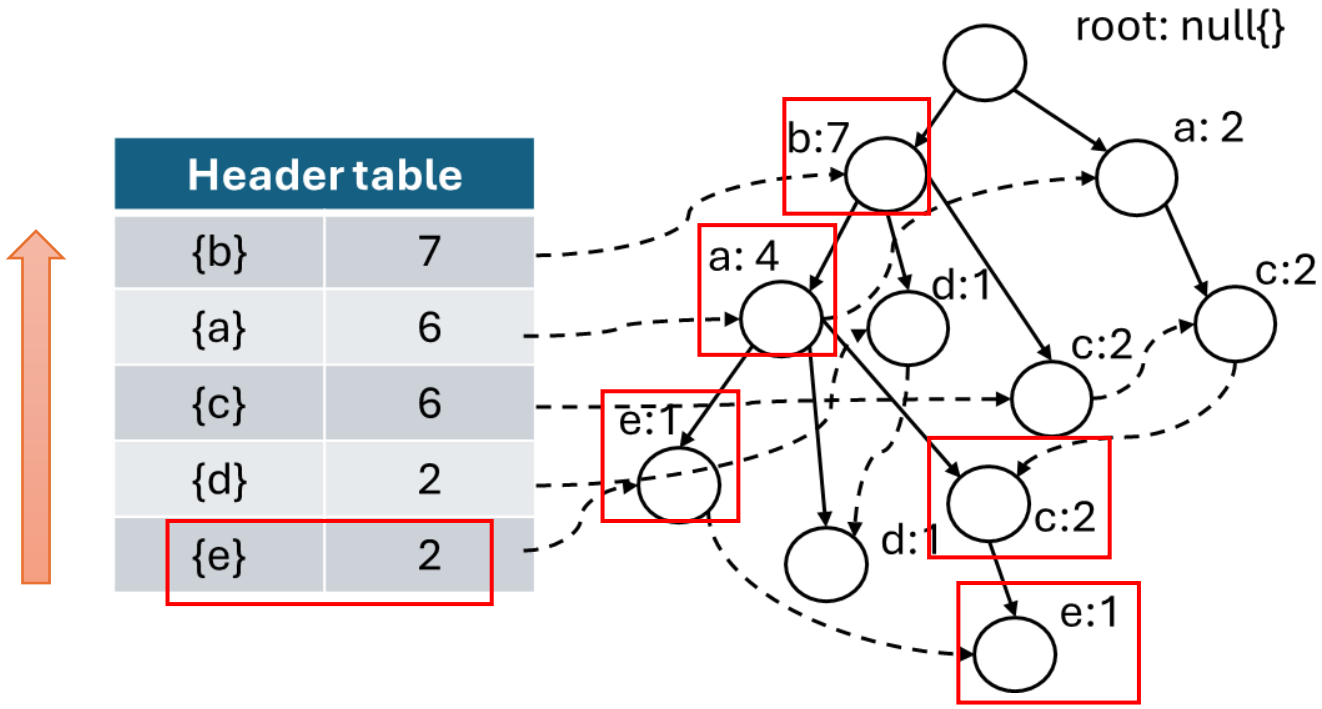
After inserting all the frequent itemlists

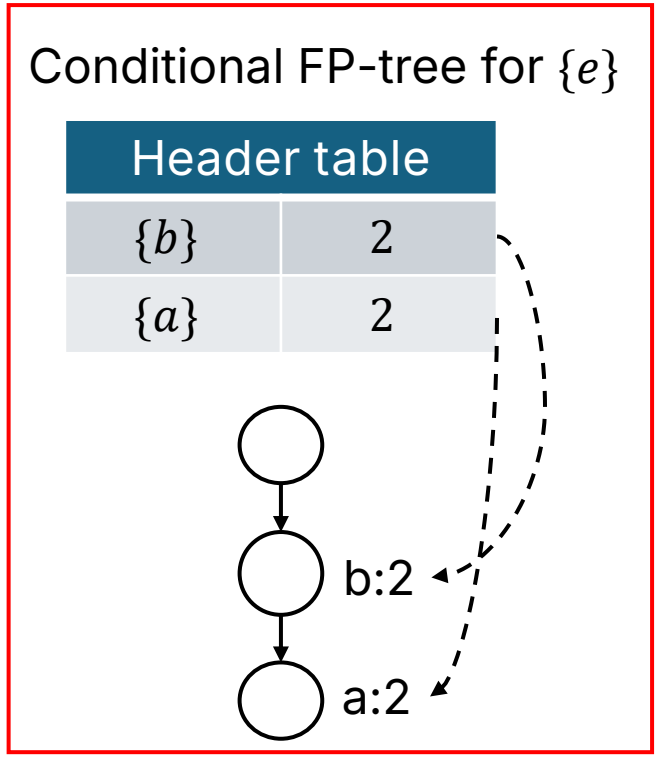
Itm	hdr
f	
c	
a	
b	
m	
p	



FP-growth: mining FP-tree

1. Begin with the 1-itemset in the f-list with the smallest count
   - An initial suffix pattern (leaf node)
2. Construct its conditional pattern base
   - A subdatabase, which consists of the set of prefix paths in the FP-tree cooccurring with the suffix pattern
3. Construct its (conditional) FP-tree, and perform mining recursively on the tree

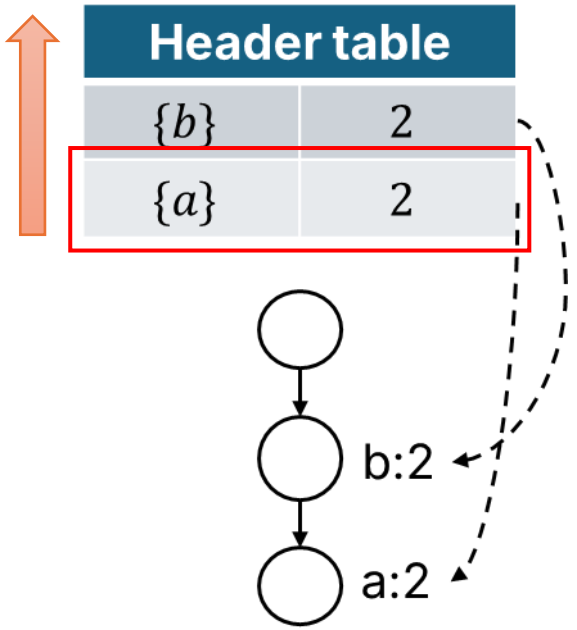


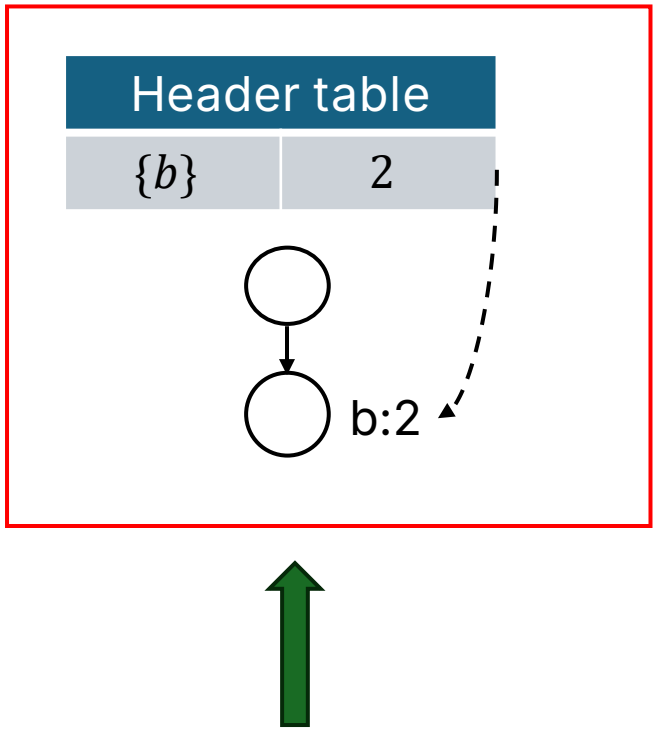


minimum support count = 2

Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{e\}$	$\{\{b, a: 1\}, \{b, a, c: 1\}\}$	$< b: 2, a: 2, \sout{c: 1} >$

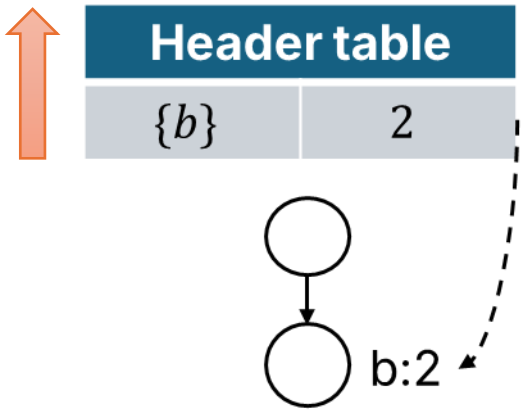
Conditional FP-tree for $\{e\}$





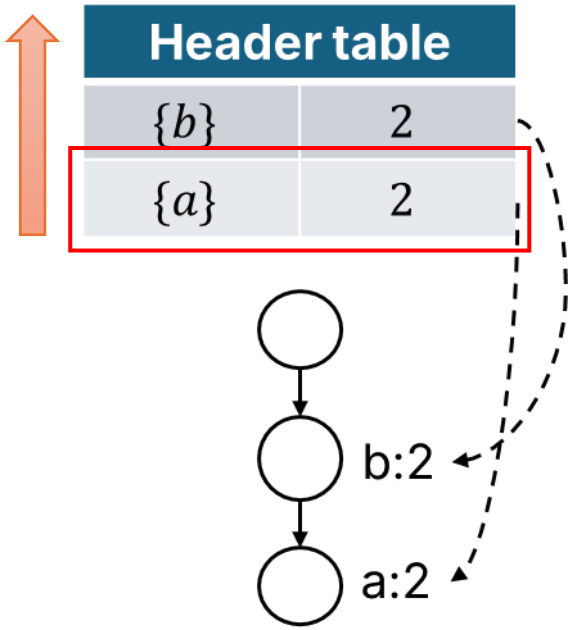
Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{a, e\}$	$\{\{b: 2\}\}$	$< b: 2 >$

Conditional FP-tree for $\{a, e\}$



Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{b, a, e\}$	$\{null\}$	$< null >$

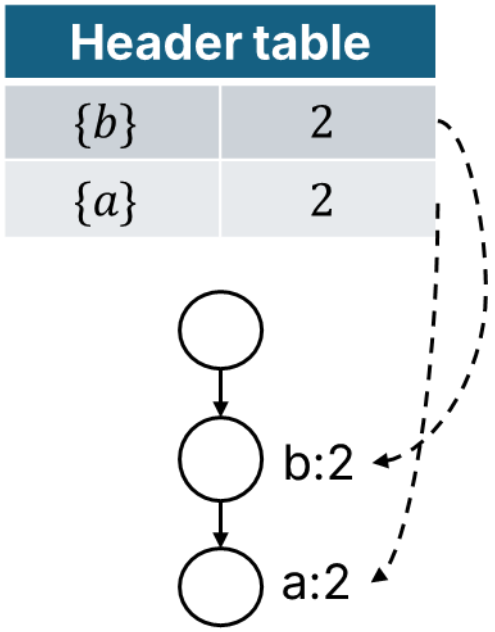
Conditional FP-tree for $\{e\}$



Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{a, e\}$	$\{\{b: 2\}\}$	$< b: 2 >$
$\{b, e\}$	$\{null\}$	$< null >$

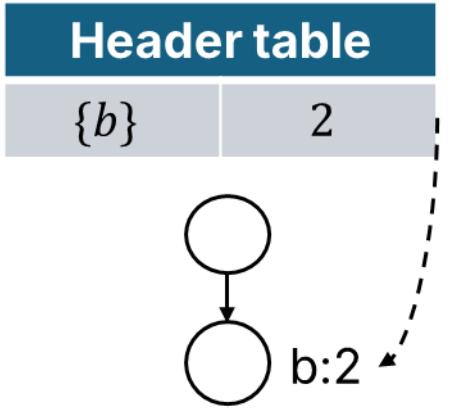
Conditional FP-tree for $\{e\}$

Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{a, e\}$	$\{\{b: 2\}\}$	$< b: 2 >$
$\{b, e\}$	$\{null\}$	$< null >$

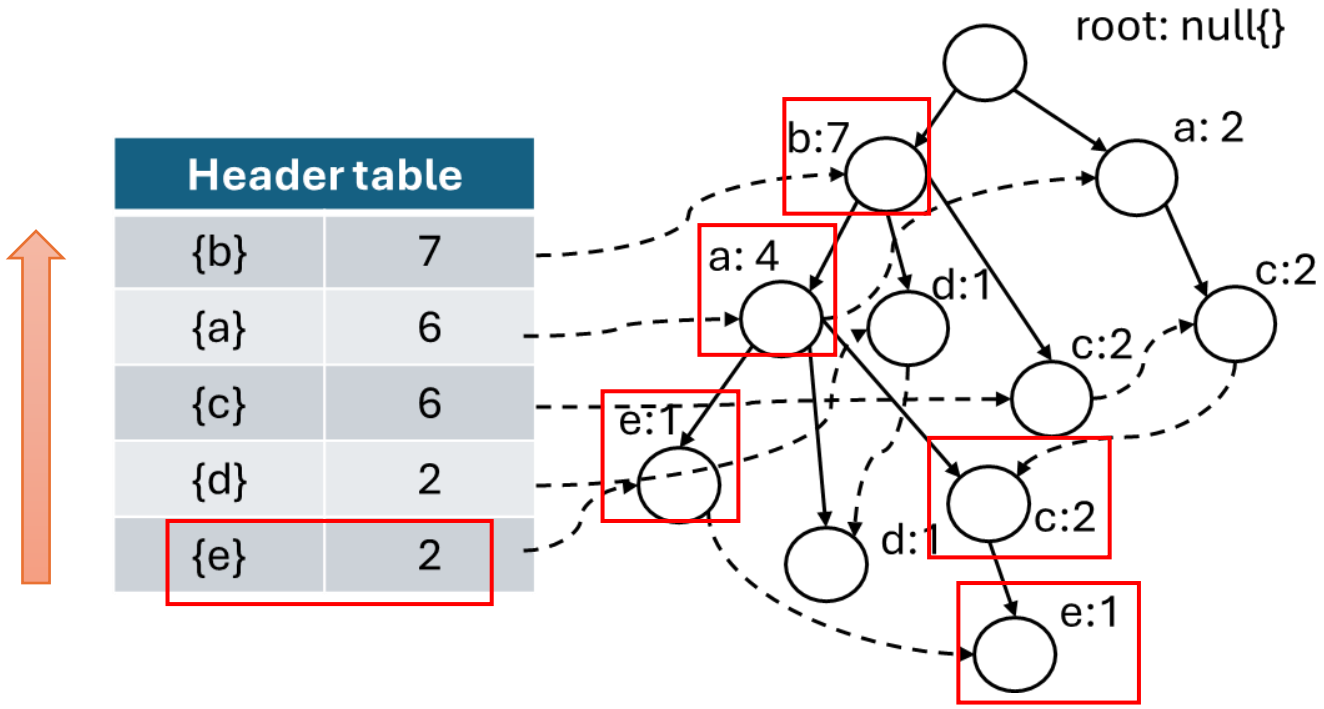


Conditional FP-tree for $\{a, e\}$

Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{b, a, e\}$	$\{null\}$	$< null >$

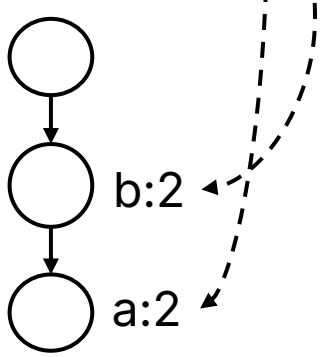


$\{e\}$



Conditional FP-tree for $\{e\}$

Header table	
$\{b\}$	2
$\{a\}$	2



Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{e\}$	$\{\{b, a: 1\}, \{b, a, c: 1\}\}$	$< b: 2, a: 2, \sout{c: 1} >$

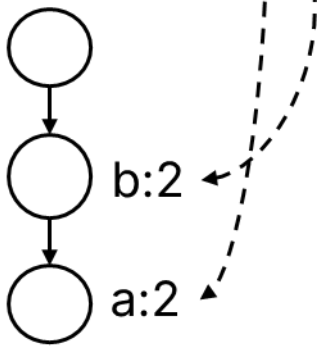
Frequent patterns involving e:

- $\{e\}: 2$
- $\{b, e\}: 2$
- $\{a, e\}: 2$
- $\{b, a, e\}: 2$

Another Solution for Single Path

Conditional FP-tree for $\{e\}$

Header table	
$\{b\}$	2
$\{a\}$	2



Single Path:

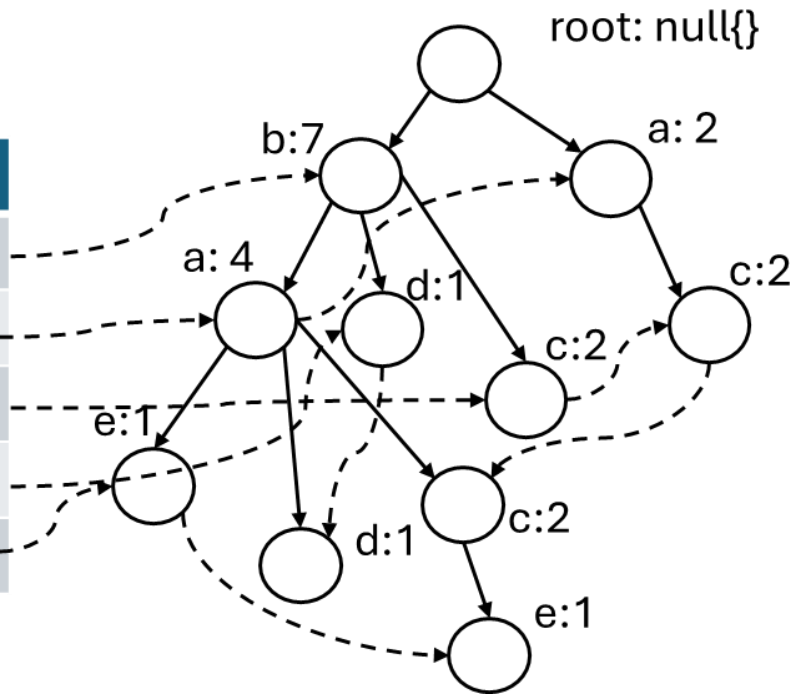
① List all the combination β of the nodes

- $\{a\}$
- $\{b\}$
- $\{b, a\}$

② Union β with the suffix $\{e\}$

- $\{a, e\}$
- $\{b, e\}$
- $\{b, a, e\}$

Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2



Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{e\}$	$\{\{b, a: 1\}, \{b, a, c: 1\}\}$	$< b: 2, a: 2 >$
$\{d\}$	$\{\{b: 1\}, \{b, a\}: 1\}$	$< b: 2 >$
$\{c\}$	$\{\{b: 2\}, \{a: 2\}, \{b, a: 2\}\}$	$< b: 4, a: 2 >, < a: 2 >$
$\{a\}$	$\{\{b: 4\}\}$	$< b: 4 >$
$\{b\}$	No prefix	

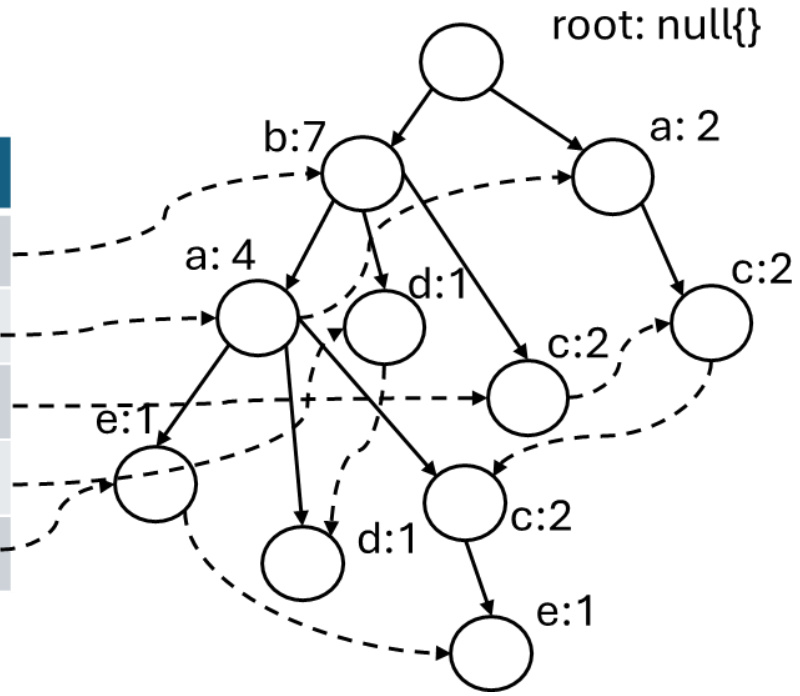
Frequent Pattern:
- {d}:2
- {b, d}:2

Frequent Pattern:
- {c}:6
- {b, c}:4
- {a, c}:4
- {b, a, c}:2

Frequent Pattern:
- {a}:6
- {b, a}:4

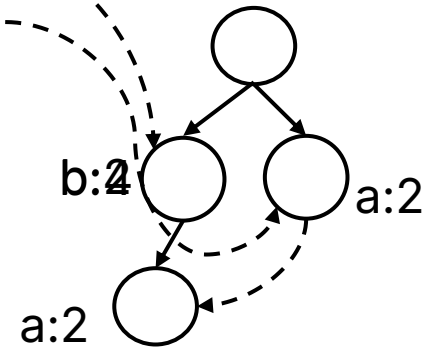
Frequent Pattern:
- {b}:7

Header table	
{b}	7
{a}	6
{c}	6
{d}	2
{e}	2



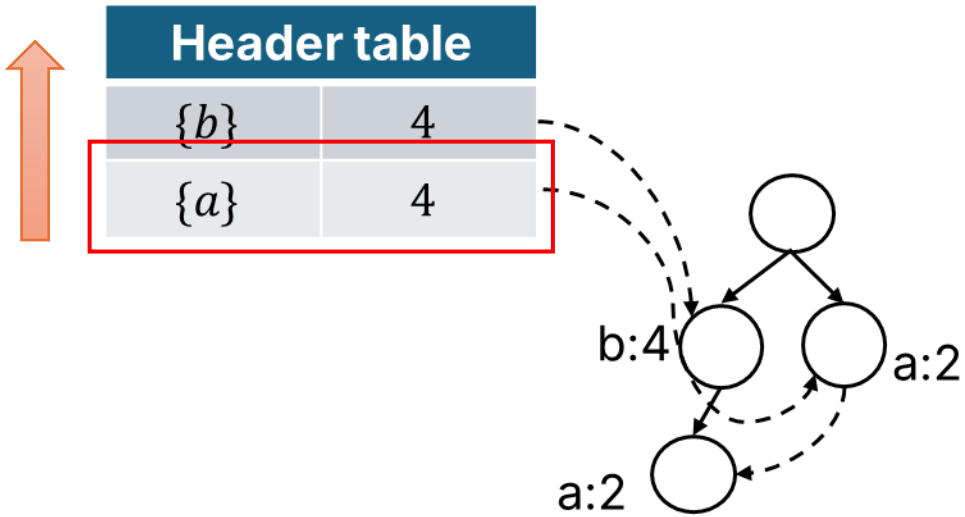
Conditional FP-tree for $\{c\}$

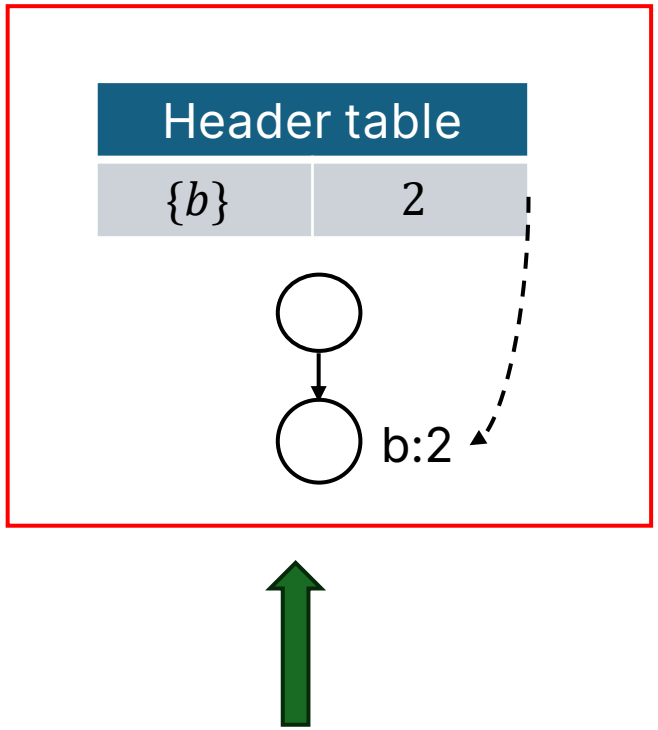
Header table	
$\{b\}$	4
$\{a\}$	4



Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{c\}$	$\{\{b{:}\ 2\},\{a{:}\ 2\},\{b,a{:}\ 2\}\}$	$< b{:}4, a{:}2 >, < a{:}2 >$

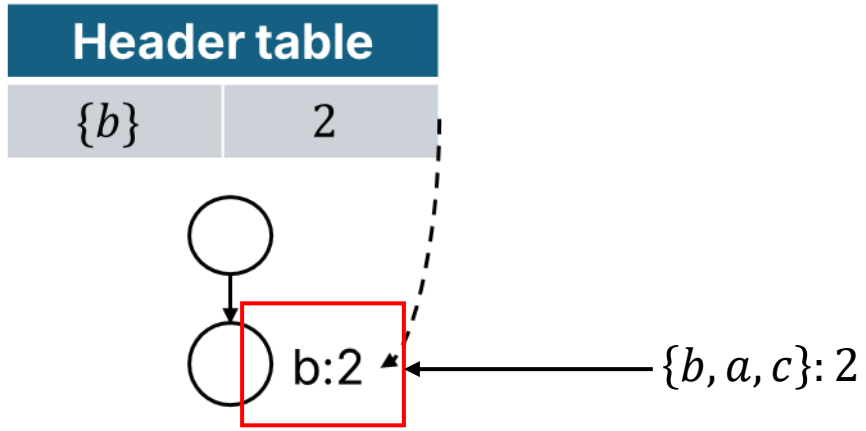
Conditional FP-tree for $\{c\}$





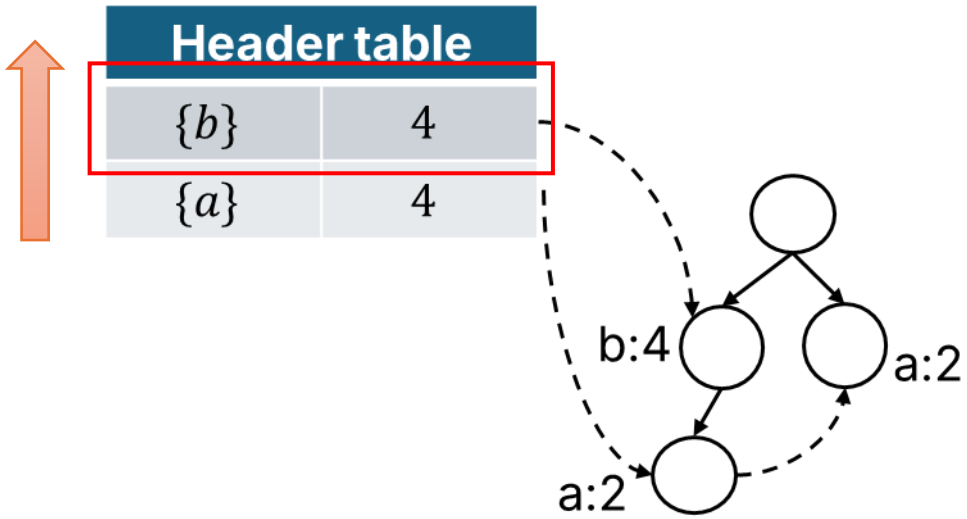
Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{a, c\}$	$\{\{b: 2\}\}$	$< b: 2 >$

Conditional FP-tree for $\{a, c\}$



Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{b, a, c\}$	$\{null\}$	$< null >$

Conditional FP-tree for $\{c\}$



Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{a, c\}$	$\{\{b: 2\}\}$	$< b: 2 >$
$\{b, c\}$	$\{null\}$	$< null >$

Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{a, c\}$	$\{\{b: 2\}\}$	$< b: 2 >$
$\{b, c\}$	$\{null\}$	$< null >$

Initial Suffix	Conditional pattern base	Conditional FP-tree
$\{b, a, c\}$	$\{null\}$	$< null >$

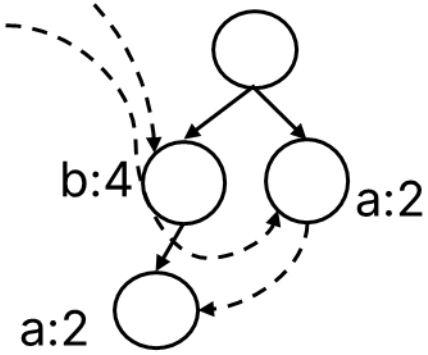
$\{c\}$

Frequent Pattern:

- **{c}:6**
- **{b, c}:4**
- **{a, c}:4**
- **{b, a, c}:2**

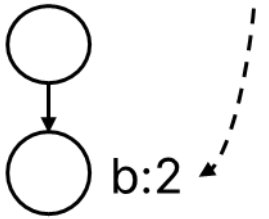
Conditional FP-tree for $\{c\}$

Header table	
$\{b\}$	4
$\{a\}$	4



Conditional FP-tree for $\{a, c\}$

Header table	
$\{b\}$	2



L

L_1

Itemset	count
$\{a\}$	6
$\{b\}$	7
$\{c\}$	6
$\{d\}$	2
$\{e\}$	2

L_2

Itemset	count
$\{a,b\}$	4
$\{a,c\}$	4
$\{a,e\}$	2
$\{b,c\}$	4
$\{b,d\}$	2
$\{b,e\}$	2

L_3

Itemset	count
$\{a,b,c\}$	2
$\{a,b,e\}$	2

suffix pattern $\{e\}$:
- $\{e\}$: 2
- $\{b,e\}$: 2
- $\{a,e\}$: 2
- $\{b,a,e\}$: 2

suffix pattern $\{d\}$:
- $\{d\}$: 2
- $\{b,d\}$: 2

suffix pattern $\{c\}$:
- $\{c\}$: 6
- $\{b,c\}$: 4
- $\{a,c\}$: 4
- $\{b,a,c\}$: 2

suffix pattern $\{a\}$:
- $\{a\}$: 6
- $\{b,a\}$: 4

suffix pattern $\{b\}$:
- $\{b\}$: 7

F-list: b-a-c-d-e

FP-tree algorithm

1. The FP-tree is constructed in the following steps:
   a. Scan the transaction database D once. Collect F, the set of frequent items, and their support counts. Sort F in support count descending order as L, the *list* of frequent items.
   b. Create the root of an FP-tree, and label it as "null." For each transaction *Trans* in D do the following.
   Select and sort the frequent items in *Trans* according to the order of L. Let the sorted frequent item list in *Trans* be $[p|P]$, where p is the first element and P is the remaining list. Call insert_tree($[p|P]$, T), which is performed as follows. If T has a child N such that $N.item\text{-}name = p.item\text{-}name$, then increment N's count by 1; else create a new node N, and let its count be 1, its parent link be linked to T, and its node-link to the nodes with the same *item-name* via the node-link structure. If P is nonempty, call insert_tree(P, N) recursively.
2. The FP-tree is mined by calling FP_growth(*FP_tree*, *null*), which is implemented as follows.

FP-growth algorithm

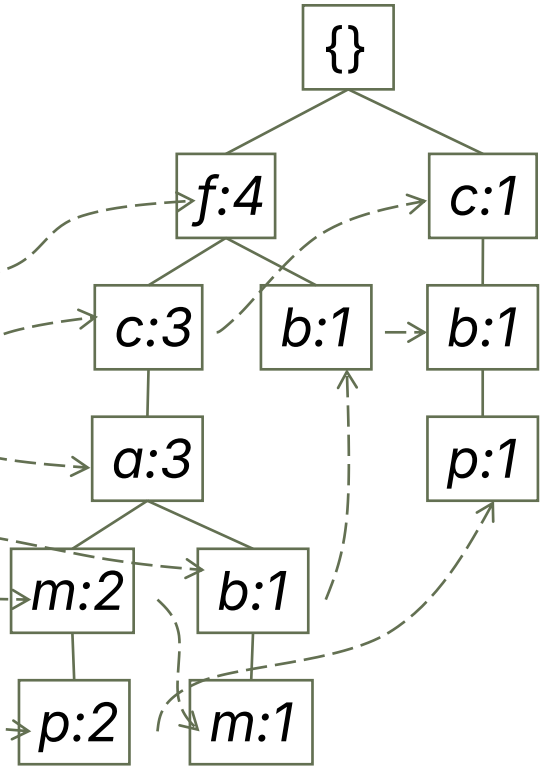
procedure FP_growth(*Tree*, α)

(1) **if** $Tree$ contains a single path P **then**
(2) **for each** combination (denoted as β) of the nodes in the path P
(3) generate pattern $\beta \cup \alpha$ with $support_count = minimum\ support\ count\ of\ nodes\ in\ \beta$;
(4) **else for each** a_i in the header of $Tree$ {
(5) generate pattern $\beta = a_i \cup \alpha$ with $support_count = a_i.support_count$;
(6) construct β's conditional pattern base and then β's conditional FP_tree $Tree_\beta$;
(7) **if** $Tree_\beta \neq \emptyset$ **then**
(8) call FP_growth($Tree_\beta$, β); }

Mining FP-Tree Example

min_support = 3

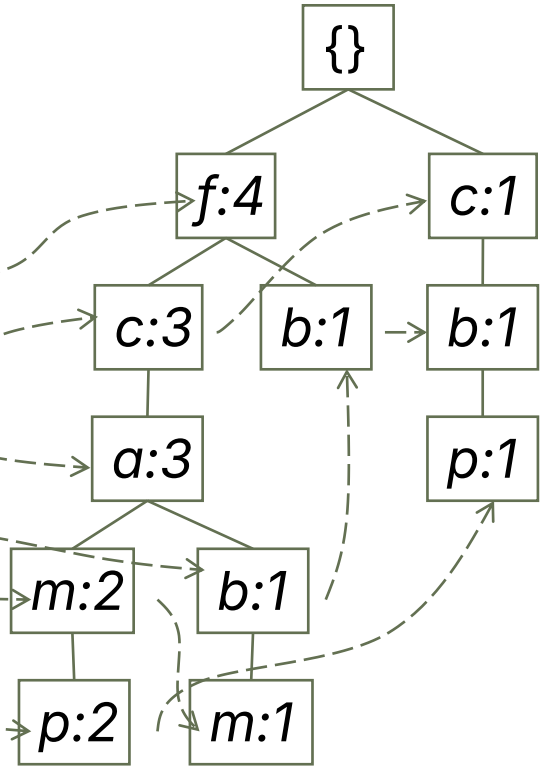
Item	Frequency	Header
f	4	
c	4	
a	3	
b	3	
m	3	
p	3	



Mining FP-Tree Example (cont.)

min_support = 3

Item	Frequency	Header
f	4	
c	4	
a	3	
b	3	
m	3	
p	3	



Conditional database of each pattern

Item	*Conditional database*
c	***f:3***
a	***fc:3***
b	***fca:1, f:1, c:1***
m	***fca:2, fcab:1***
p	***fcam:2, cb:1***

Mining FP-Tree Example (cont.)

min_support = 3

Conditional database of each pattern

Item	*Conditional database*
c	*f:3*
a	*fc:3*
b	*fca:1, f:1, c:1*
m	*fca:2, fcab:1*
p	*fcam:2, cb:1*

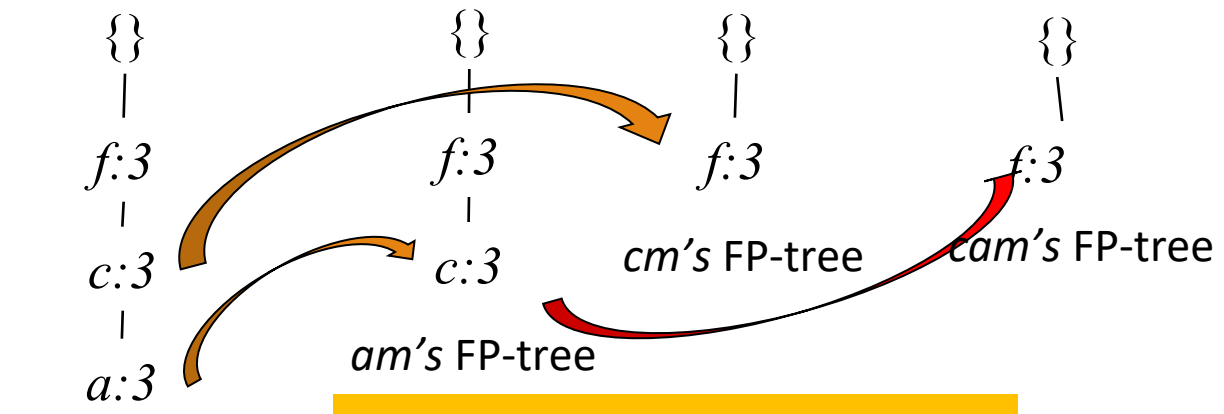
p's conditional DB: ***fcam:2, cb:1 → c: 3***

m's conditional DB: ***fca:2, fcab:1 → fca: 3***

b's conditional DB: ***fca:1, f:1, c:1 → ϕ***

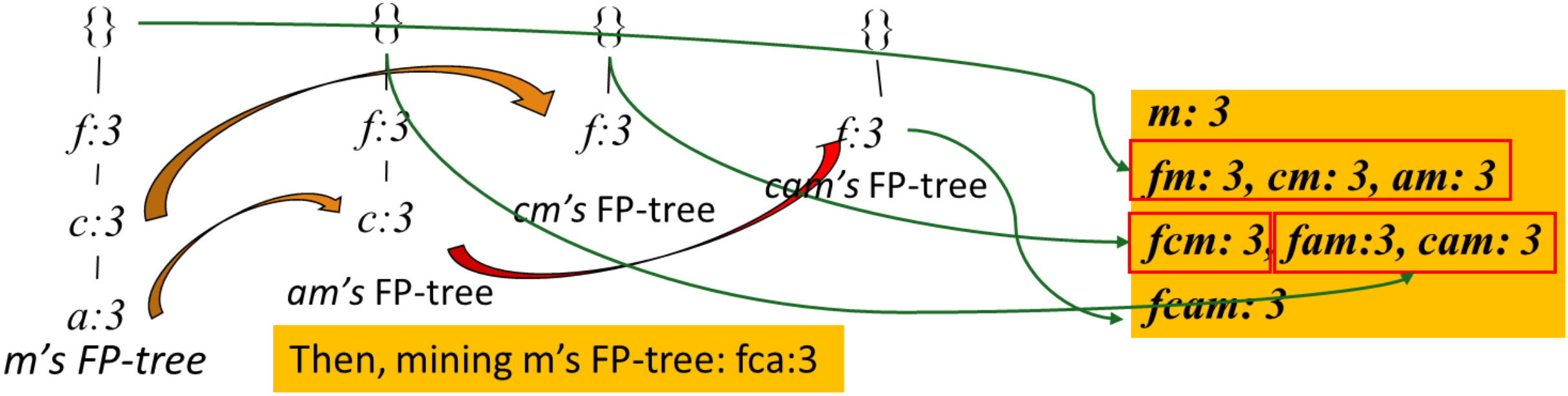
a's conditional DB: ***fc:3***

c's conditional DB: ***f:3***



Then, mining m's FP-tree: fca:3

Mining FP-Tree Example (cont.)



Quantitative attributes

Quantitative Association Rules

- Association rule mining from relational data (attributed data) with quantitative attributes (numerical attributes, interval attributes)
  - Numerical attributes: age and salary
- Approach: transform into transaction data
  1. Discretization numerical attributes using discretization methods or concept hierarchies.
  2. Transform each categorical value or discretized interval into an item.

Quantitative Association Rules (cont.)

- Binning methods
  - Equal-width: divide the value range into intervals of equal width.
  - Equal-frequency (equal-depth): each interval contains approximately the same number of values.
  - Clustering-based: group similar values into bins using a clustering method.

Price($)	Equal-width	Equal-depth	Distance-based
7	[0, 10]	[7, 20]	[7]
20	[11, 20]	[22, 50]	[20, 22]
22	[21, 30]	[51, 53]	[50, 51, 53]
50	[31, 40]		
51	[41, 50]		
53	[51, 60]		

Which patterns are interesting?

Pattern evaluation

Interestingness

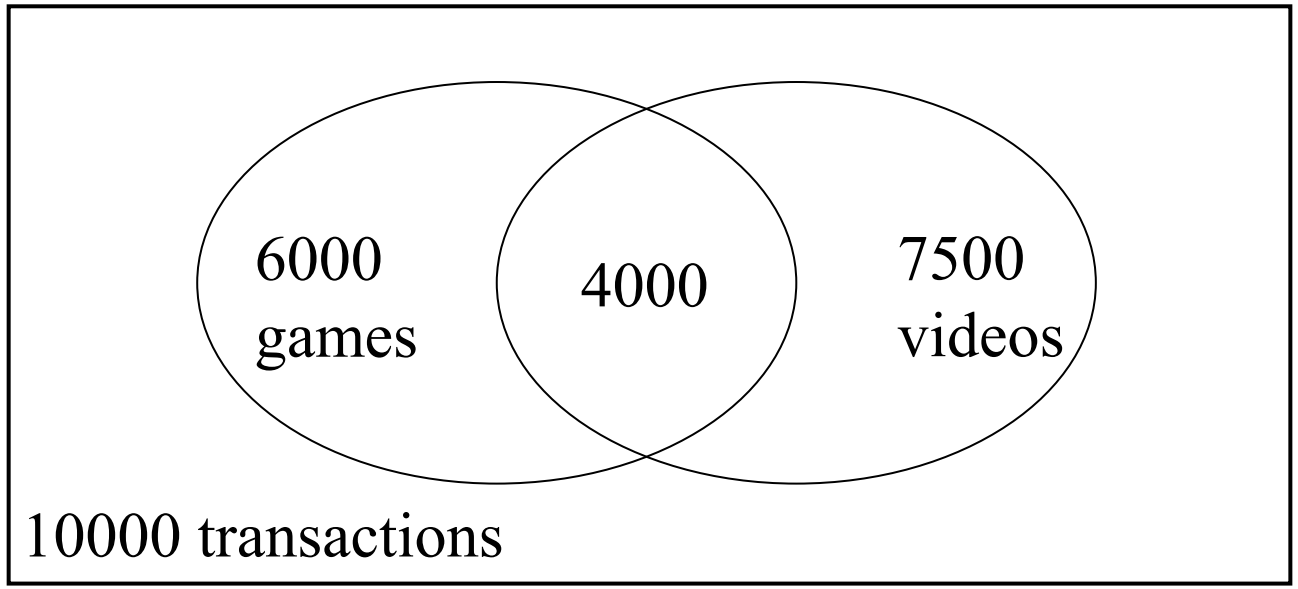
- A rule’s interestingness has both subjective and objective aspects.
  - ① Subjective interestingness depends on the user’s goals and may differ from one user to another.
  - ② Objective interestingness measures provide statistical evidence for prioritizing or filtering rules.
- Objective measures support, but do not replace, the user’s final judgment.

Pattern evaluation

- Although minimum support and confidence thresholds help filter out many uninteresting rules, many generated rules may still not be interesting to users.
  - This problem becomes more severe when mining with low support thresholds or searching for long patterns.
- Correlation analysis
  - How the support–confidence framework can be supplemented with additional interestingness measures.

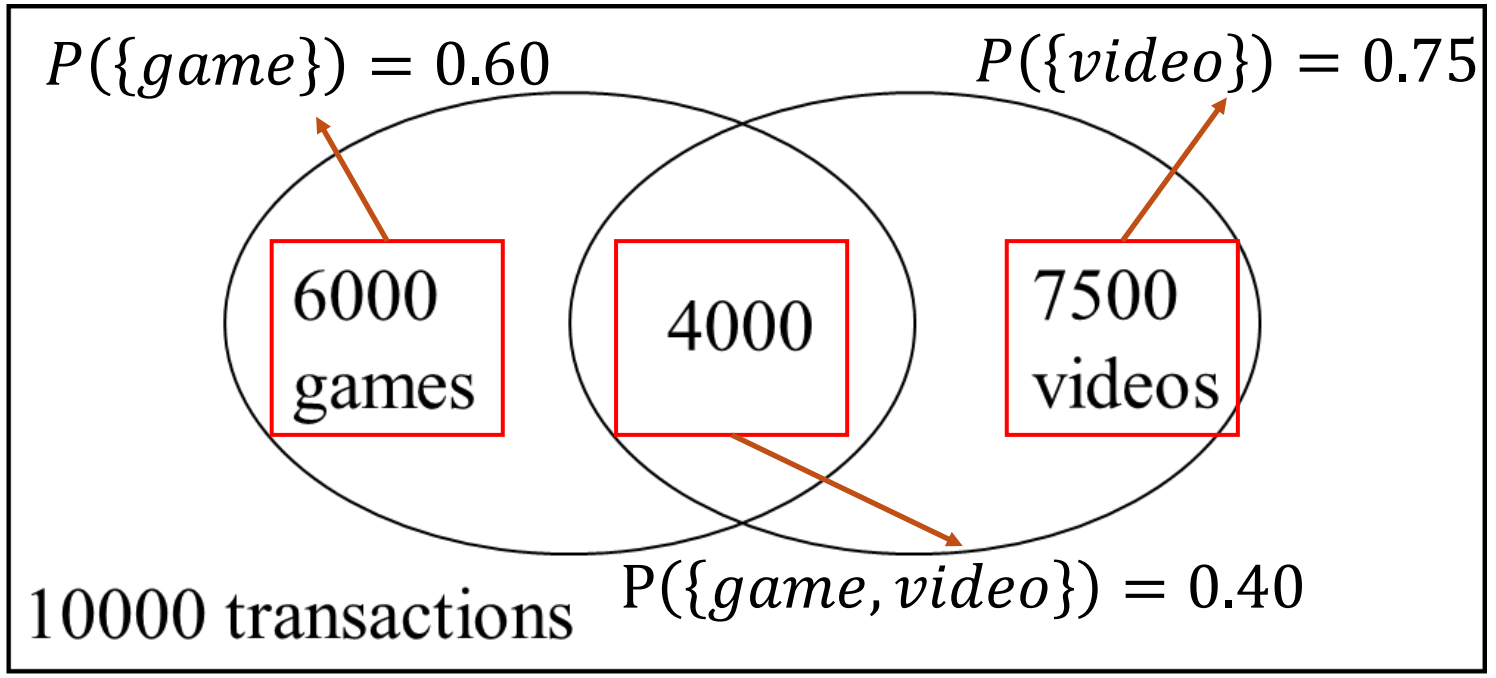
Strong rules are not necessarily interesting

- How can we tell which strong association rules are really interesting?



$\{game\} \rightarrow \{video\}$, support: $\frac{4000}{10000} = 40\%$, confidence: $\frac{4000}{6000} = 66\%$

From association analysis to correlation analysis



$$conf(\{game\} \rightarrow \{video\}) =$$
$$P(video|game) = \frac{4000}{6000} = 0.67$$
$$< P(\{video\}) = 0.75$$

Should we consider the support of the consequent?

	Game	Not Game	Total
Video	4000	3500	7500
Not Video	2000	500	2500
Total	6000	4000	10000

$$\{not\ game\} \rightarrow \{video\} = \frac{3500}{4000}$$

$$\{not\ video\} \rightarrow \{game\} = \frac{2000}{2500}$$

Strong rules are not necessarily interesting

- In fact, games & videos are negatively associated
- Transactions containing games have a lower probability of containing videos than the overall baseline.
- What kind of strong rules do we really want?
  - $Confidence(\mathrm{X} \rightarrow Y)$ should be sufficiently high
    - To ensure that transactions containing X are more likely to contain Y than a random transaction.
  - $Confidence(X \rightarrow Y) > support(Y)$
    - If $confidence(X \rightarrow Y) < support(Y)$, transactions containing X are less likely to contain Y than a random transaction.

Strong rules are not necessarily interesting

$$confidence(X \rightarrow Y) = P(Y|X)$$

$$confidence(X \rightarrow Y) \begin{cases} > support(Y), positive\ correlation \\ = support(Y), independence \\ < support(Y), negative\ correlation \end{cases}$$

$$P(Y|X) = \frac{P(X \cap Y)}{P(X)} \begin{cases} > P(Y) \\ = P(Y) \\ > P(Y) \end{cases}$$

Lift (correlation analysis)

- A simple correlation measure
  - The occurrence of itemset A is independent of the occurrence of itemset B if $P(A \cap B) = P(A)P(B)$
  - Otherwise, itemsets A and B are dependent and correlated

$$confidence(X \rightarrow Y) = \frac{P(X \cap Y)}{P(X)} \quad lift(X,Y) = \frac{P(X \cap Y)}{P(X)P(Y)} = \frac{conf(X \rightarrow Y)}{sup(Y)} = \frac{sup(X \cup Y)}{\sup(X)sup(Y)}$$

	Game	Not Game	Total
Video	4000	3500	7500
Not Video	2000	500	2500
Total	6000	4000	10000

$$lift(game, video) = \frac{\frac{4000}{10000}}{\frac{7500}{10000} \times \frac{6000}{10000}} = 0.89$$

- $lift(X,Y) = 1$, X and Y are independent
- $lift(X,Y) < 1$, X and Y are negatively associated.
- $lift(X,Y) > 1$, X and Y are positively associated.

From association analysis to correlation analysis (cont.)

- An association rule $X \rightarrow Y$ should be evaluated not only by support and confidence, but also by an interestingness measure of the association between A and B, such as lift.

$$\{X\} \rightarrow \{Y\}, [\text{support}, \text{confidence}, \text{correlation}]$$

Other measures to evaluate rules

- Chi-square measure

$$\chi^2 = \Sigma \frac{(observed - expected)^2}{expected}$$

- All-confidence measure

$$all_conf(A, B) = \frac{sup(A \cup B)}{max\{sup(A), sup(B)\}}$$

- Max-confidence measure

$$max_conf(A, B) = max\{P(A \mid B), P(B \mid A)\}$$

- Kulczynski measure

$$Kulc(A, B) = \frac{1}{2}(P(A|B) + P(B|A))$$

- Cosine measure

$$cosine(A, B) = \frac{P(A \cap B)}{\sqrt{P(A) \times P(B)}} = \frac{sup(A \cup B)}{\sqrt{sup(A) \times sup(B)}}$$
$$= \sqrt{P(A|B) \times P(B|A)}.$$

Summary

- Basic Concepts
  - What is pattern discovery?
  - Why is it important?
  - Frequent patterns and association rules
    - Maximal and closet frequent patterns
- Efficient Pattern Mining Methods
  - The Apriori algorithm
  - FP-Growth
- Pattern Evaluation
  - Lift